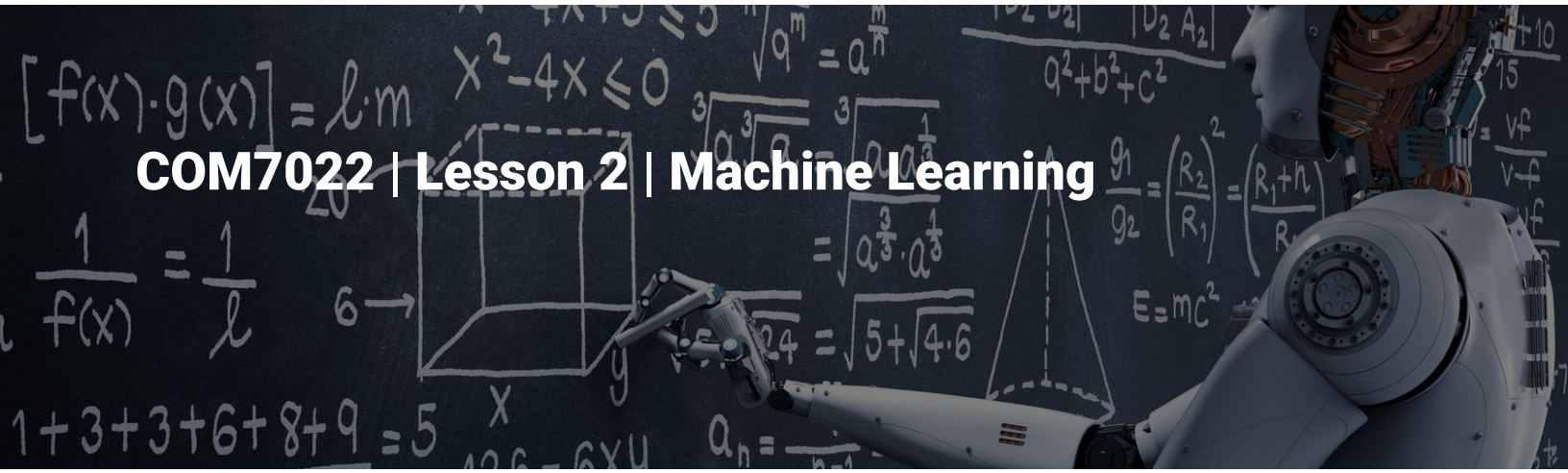


COM7022 | Lesson 2 | Machine Learning

Machine Learning (COM7022)

Lesson 2: Data Preparation

Cover image by Phonlamai Photo from Shutterstock.com

- **Lesson introduction**
- **Before you get started**
- **Topic 1: Understanding of data structure and types**
- **Topic 2: Data cleaning**
- **Topic 3: Data integration**
- **Topic 4: Data reduction**
- **Topic 5: Data transformation**

Knowledge check

Lesson summary

Reading List

Lesson introduction

Welcome to Lesson 2

LESSON INTRODUCTION | LEARNING OUTCOMES | ACCESSIBILITY GUIDANCE

Welcome to Lesson 2!

Data preparation is an important step in a data analysis/mining life cycle. As the saying goes: “Garbage in, garbage out.” If the data are not appropriately pre-processed, implementing a machine-learning model will result in undesirable and false results, providing us with false insights into the data when we apply our machine-learning models. Therefore, we should examine this process stage to ensure that you can pre-process your data well.

The following are the topics that we will cover in this lesson:

- **Topic 1 - Data structure and representation**
  This topic will provide you with an overall understanding of the data type you will be using and the data representation within a dataset.
- **Topic 2 - Data cleaning**
  This topic will examine the typical issues in a sample raw data extracted from a database, enabling you to understand the issues, and to learn and implement the techniques and practical functions to clean the data.

- **Topic 3 - integration**
  We will investigate the functions of data integration and how we can do this with our data.
- **Topic 4 - Data reduction**
  This topic will introduce you to some techniques and allow you to implement these techniques with a dataset.
- **Topic 5- Data transformation**
  Finally, we will investigate the process of data transformation. This process will enable you to transform your data while maintaining value in terms of the insights the data will provide.

There is no doubt this lesson is a big step forward compared with Lesson 1, but we need to have a good grasp of the topics. Remember to feel free to repeat any topic you are not sure about and to build on the cheat sheet we have been creating since Lesson 1 on the functions we will encounter in this lesson.

By the end of this lesson, you will:

- Have a good understanding of the type of data you will be processing
- Know what data cleaning is and the practical functions available in Python to carry out this process
- Have a critical understanding of the importance of data integration.
- Have a full understanding of data reduction and how this process will support you when dealing with large datasets
- Know how data transformation is implemented.

Based on the above, Lesson 2 aligns with Learning Objective 2: **Select and apply appropriate machine learning techniques for a given scenario**.

Computing

At Arden University

We have worked across teams at Arden University to try and create an accessible and inclusive experience within the learning content. We are continuously assessing the accessibility of our lessons to make improvements and enhancements based on testing and feedback.

This lesson may include screenshots of code and software used. Please be aware that these may not always have a text alternative, and that other features such as size and colour may not be accessible.

If you experience any issues with accessing the learning content throughout this module, there are several support services in place to help you.

- Arden University has a dedicated Inclusion Services Team who can help you with getting adjustments and equipment.
- The Library Portal also has information on the accessibility tools that can help you access resources: Accessibility Tools – Library Portal
- If you are struggling with accessing a specific part of the content e.g. a video resource that is missing captions, please speak to your Lecturer who may be able to advise on an alternative resource in the interim to ensure you are able to get the learning you need to progress with the lesson.

Continue to Before you get started

Before you get started

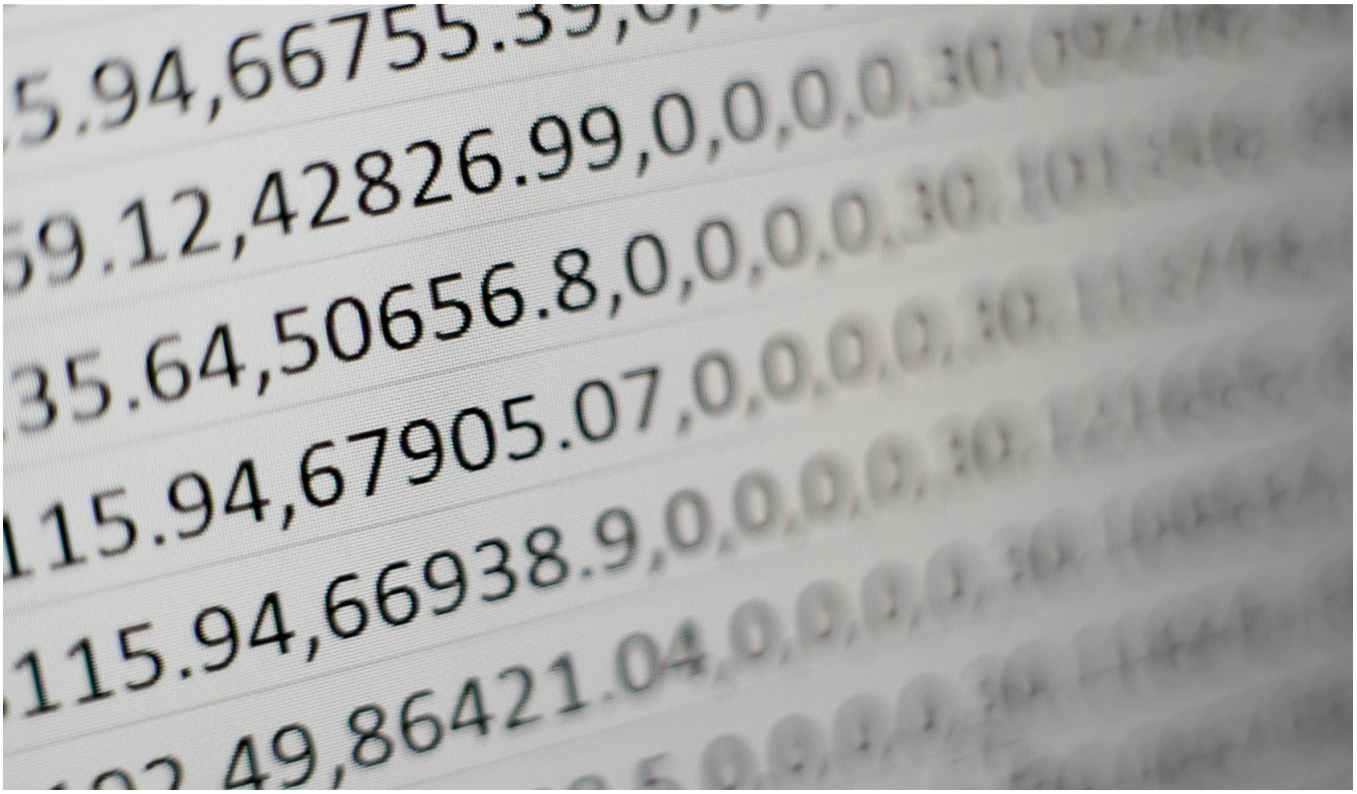


Image from Articulate Rise 360 Content Library.

Did you know?

Image from Articulate Rise 360 Content Library.

A data scientist usually spends 80% of the time preparing the dataset before designing and implementing machine models (Rençberoğlu, 2019).

Continue

Topic 2 - Activity 1

Read Lesson 8 of COM7024 - Programming for Data Analytics before the start of Topic 1.

Continue to Topic 1

Topic 1: Understanding of data structure and types

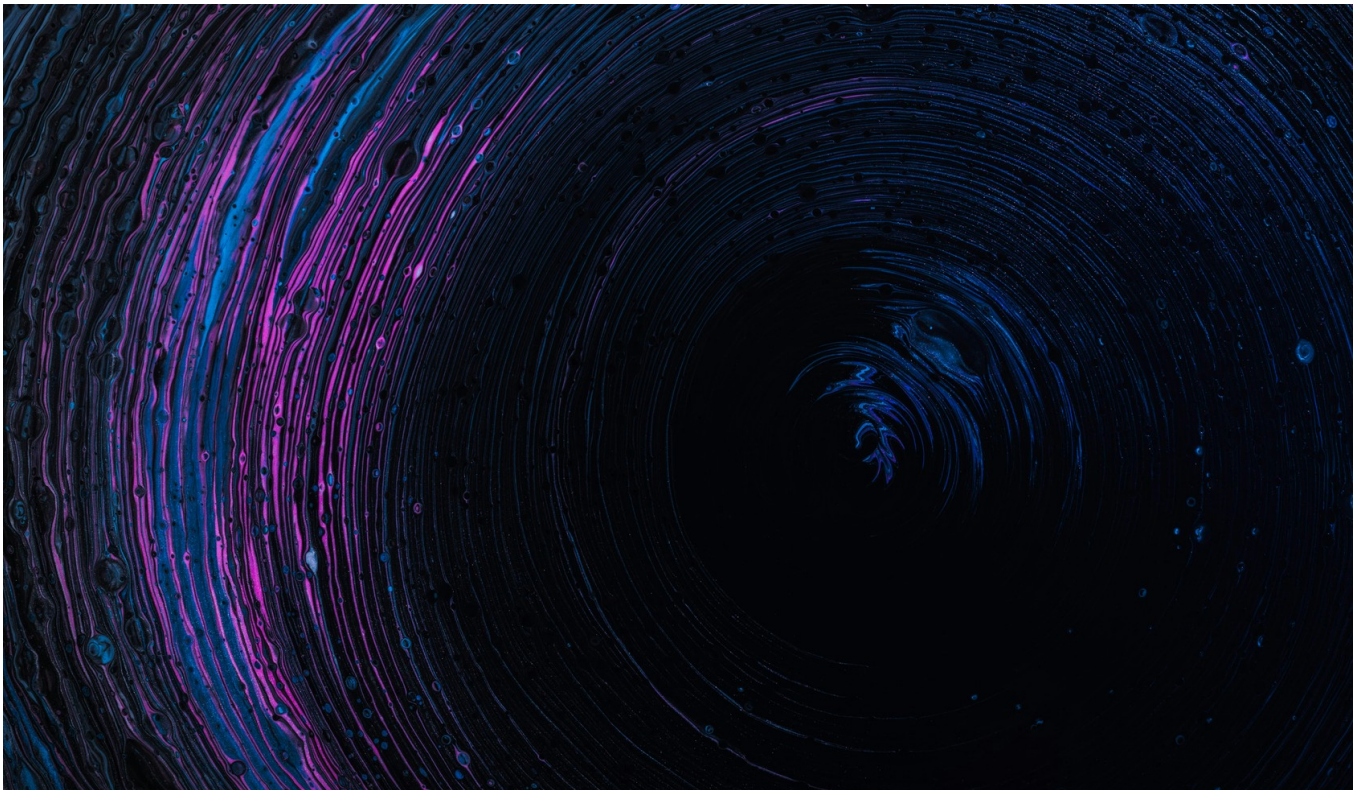


Image from Articulate Rise 360 Content Library.

Before designing and implementing machine-learning models on a particular dataset, we must understand the dataset we have been given. One of the first steps is understanding the dataset's data structure and representations.

Why do we need to understand the data structure and representation?

Continue

There are several reasons why we need to have a close examination:

1. Data types type allows us to determine the appropriate data collection. For example, if a number has been extracted and has a data type of string (a string of number characters), then we will need to consider the appropriate steps in data transformation.

2. It allows us to identify any errors in the data collection.

3. It allows us to identify the steps needed in preparation for a particular machine-learning algorithm. This is because each machine-learning algorithm requires different inputs.

4. It provides the opportunity to identify the relationship between attributes and to conduct data reduction and transformation if needed. For example, we might want to consider creating a new attribute called area for a dataset that consists of attributes (columns), width and length.

Let us start the topic by reviewing some of the terminology about the dataset, particularly the dataframe (table) we will use throughout the module. See the following table or dataframe in Python:

Row No	Attribute 1	Attribute 2	Attribute 3
1	1	2	3
2	4	5	6

Figure 2.01. Dataset terminology by Arden University.

In a Pandas dataframe, each row of a dataframe is referred to as an instance of the dataset. Each column of the dataframe is called an attribute of the dataset. From now on, we will use these two terminologies when we refer to the rows and columns of a dataframe/table.

Continue

Image from Articulate Rise 360 Content Library.

When we deal with a dataframe of a dataset, we may encounter different data types. In this topic, we will investigate each data type.

1. Qualitative (categorical) data type

1.1 Nominal data type

Nominal data types are usually called label variables without any quantitative values attached to them. They usually serve as a label or names without a dataset. For example, an animal or weather outlook attribute may have the following instance values:

- Animal = {cat, fish, dog, squirrel, lion}
- Outlook = {sunny, overcast, raining, windy}

If we look into each instance value, we cannot perform any ordering or distance measure between them. With this type of data, the only test we can perform is the equality test – i.e., “Is a dog equal to a fish?”, “Is Animal 1 equal to a cat?” with the expected response of either a Yes or a No.

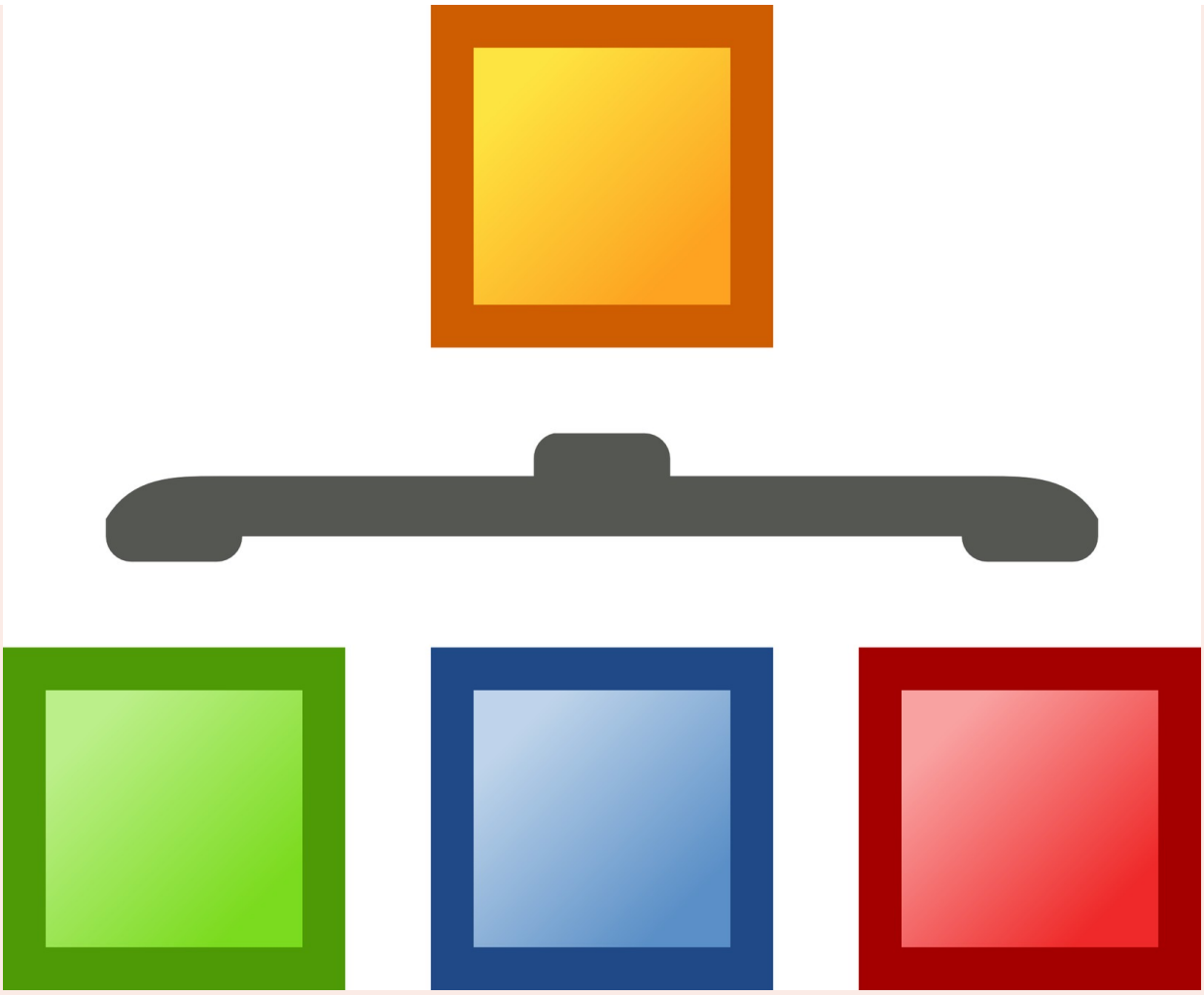


Image from Articulate Rise 360 Content Library.

1.2 Ordinal data type

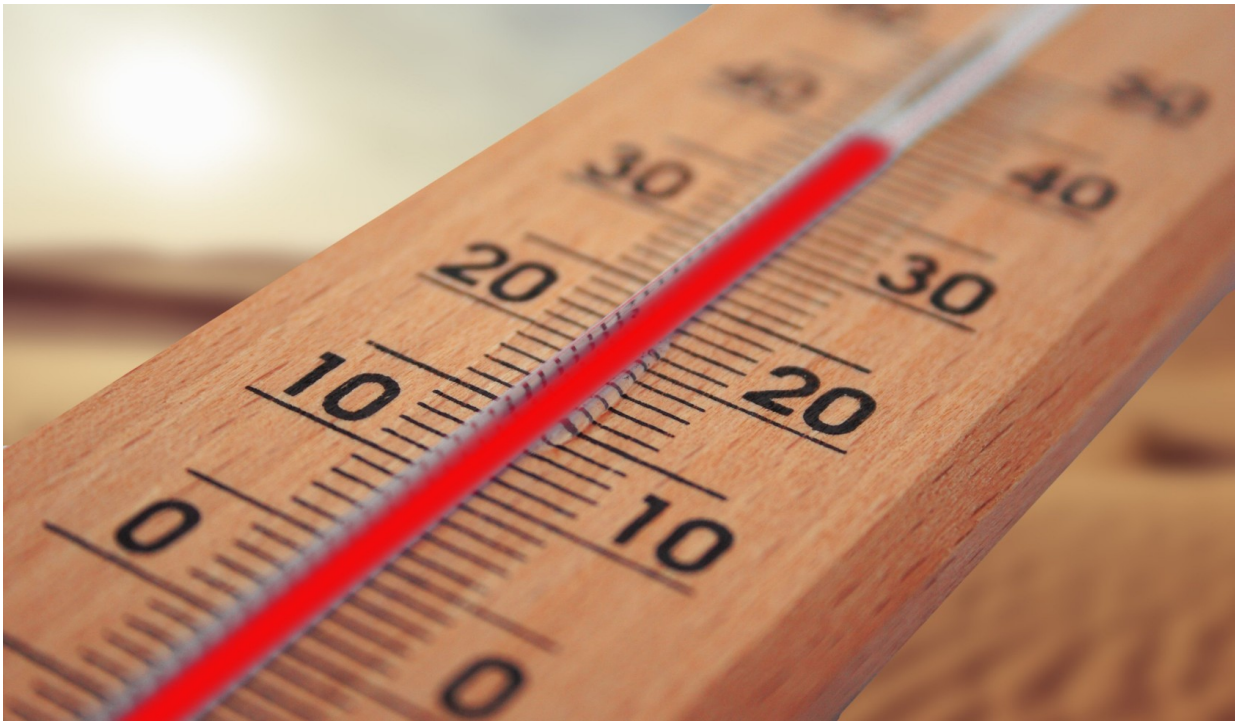


Image from Articulate Rise 360 Content Library.

Just like a nominal data type, an ordinal data instance may, at first glance, look like a label variable; however, ordinal data instances can provide more information in comparison with a nominal data instance. For example, we have the following attributes:

- Recommendation = {very likely, likely, neutral, unlikely, very unlikely}
- Temperature = {hot, mild, cool, freezing}

> Let's examine each of the examples in turn. We can notice that the instance values are orderable, in that the values can be arranged in order – that is, from high temperature to low temperature, from very likely to recommend to very unlikely to recommend, and vice versa. In both cases, to understand the ordering, we, as data scientists, will need to understand the data domain since these are domain-specific ordering.
>
> Although orders can be defined, an ordinal attribute has no distance defined between its instance values. For example:

- Temperature (values): “hot” > “mild” > “cool” > “freezing”

Based on our domain knowledge, we know that “hot” will have a higher temperature than “mild”, “mild” will have a higher temperature than “cool”, and so on. But, due to the labelling given for each instance value, we cannot qualitatively define the distance between the values.

2. Quantitative data type

Although useful, nominal and ordinal data types do not allow us to perform quantitative measurement analysis. In this section, we will examine two quantitative (numerical) data types: interval and ratio data types. These allow us to do some quantitative measurements.

2.1 Interval data type

As the name suggests, interval data allows us to examine the order of the instance values and their value differences. For example:

- Temperature= {10, 20, 30, 40}
- Temperature (values): 10 < 20< 30 < 40

The values are ordered and measured in fixed and equal units in this example. It makes sense if we look closely into the differences between the values.

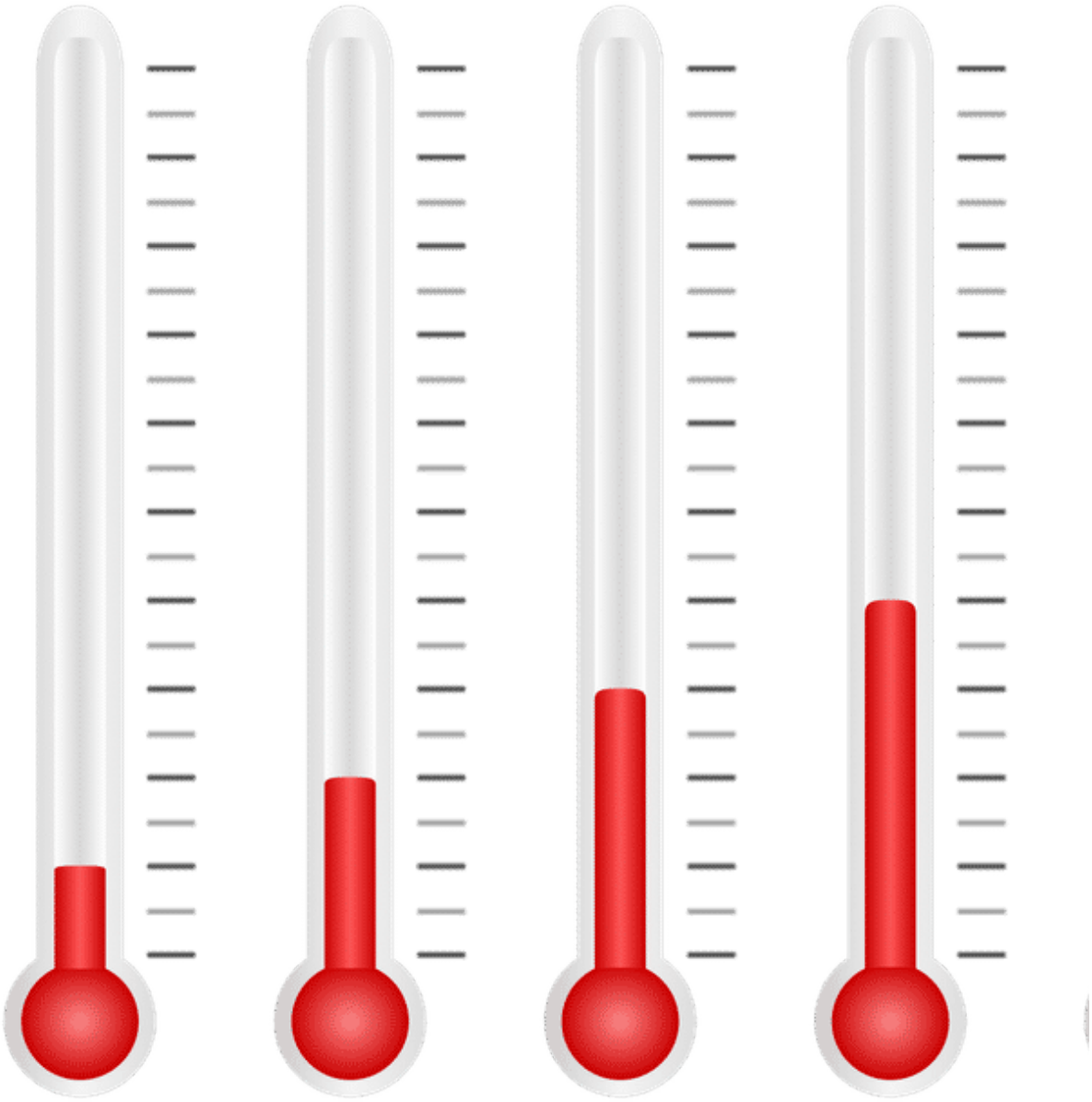


Image from Articulate Rise 360 Content Library.

However, there is a little problem with interval data. There is no “absolute zero” or point of reference since the values are measured based on the interval between the instance value and are not relative to zero. In this case, we can receive positive and negative temperatures in Celsius or Fahrenheit but not zero, because 0°C and 0°F cannot be read on the thermometer. Hence, we can use the interval value to add and subtract instance values, but we cannot do divisions and multiplications.

2.2 Ratio data type

Sometimes, it is impossible to have negative values in the attributes – for example, height and salary, because it is impossible to have a negative height or a negative salary. In this case, we label the data type as ratio data type. Ratio data are ordered, there are clear differences between them, and values are relative to zero.

Image from Articulate Rise 360 Content Library.

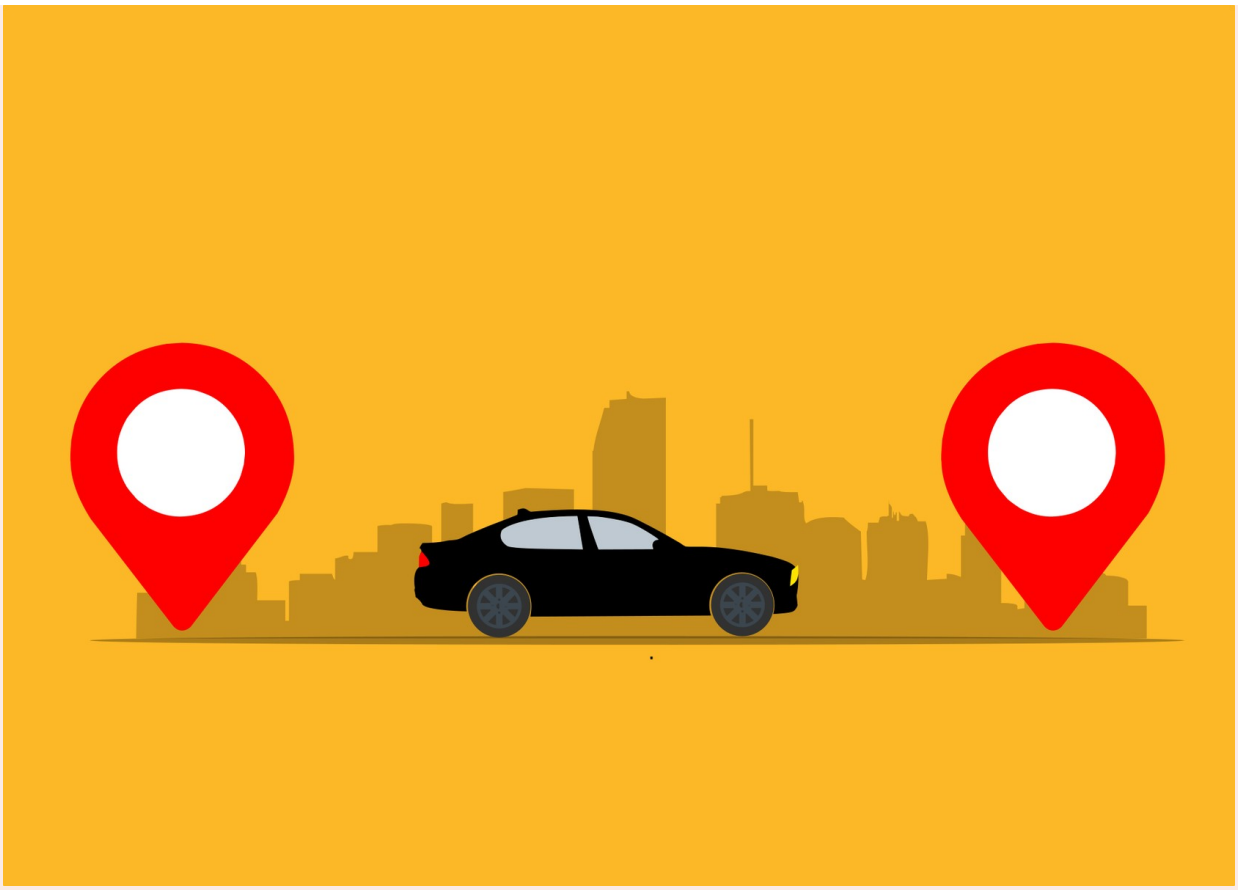


Image from Articulate Rise 360 Content Library.

For example, a car travelling 0 kilometres means that the car has not been moving. However, if a car has travelled 100 kilometres, we know it has travelled twice the distance of 50 kilometres. Once again, we know that negative values are impossible.

Continue

Numerical data values

Image from Articulate Rise 360 Content Library.

Now that we understand the attribute data type we will encounter during this module, we can further explore the numerical data values we will use within the module or in the industrial context.

1. Discrete numerical data

When we refer to discrete numerical data, we refer to a whole number with finite values. For example:

- Number of employees within a company
- Total number of students within a group
- Total number of players within a football squad

We might notice that discrete numerical data is usually used when data cannot be broken into decimal points. In the case of the examples above, we cannot split a person into decimal points to fit the narrative we

want.

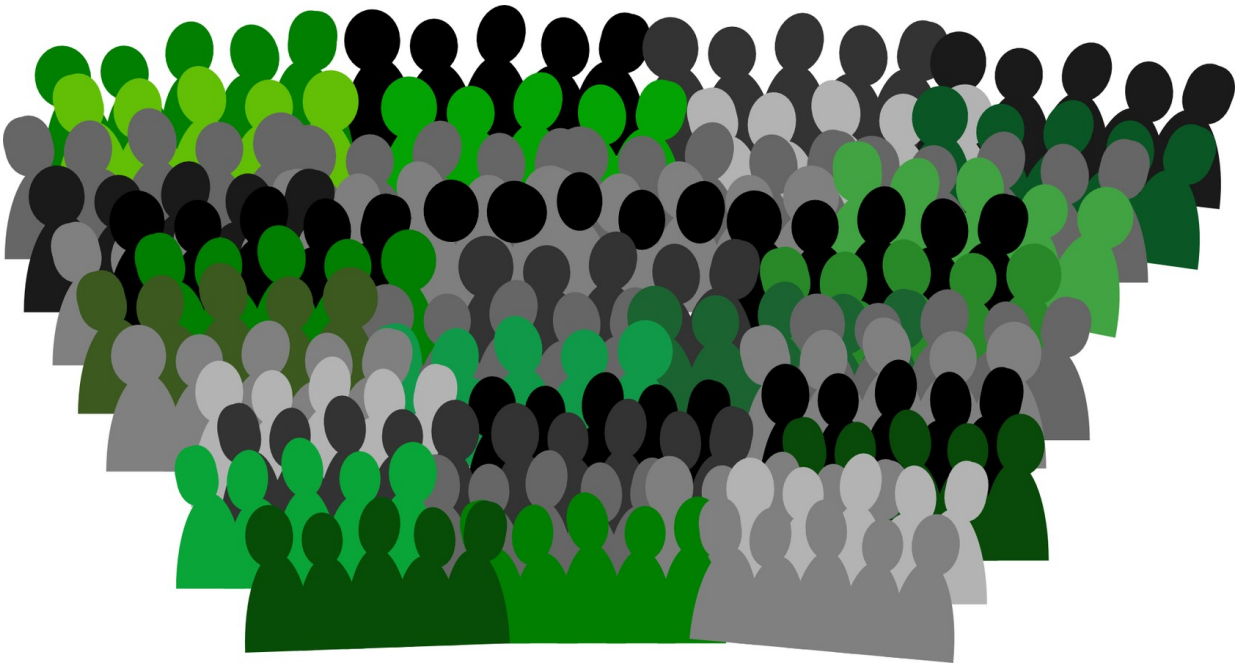


Image from Articulate Rise 360 Content Library.

---

2. Continuous numerical data

Image from Articulate Rise 360 Content Library.

Continuous data is in the form of a combination of whole and decimal or fraction numbers. This allows us to collect or record more detailed attribute data points within a certain range. For example:

- Temperature of the days within a year
- Speed of a vehicle
- House price
- Salary

3. Time series data

As the name suggests, time series data is a unique collection based on a certain measurement over time. In this data, the sequence of the instances is vital in the analysis using a machine-learning algorithm. Some examples of time series data are:

- Share prices
- ECG measurement over time
- Covid hospitalisation
- Rainfall over a period of days

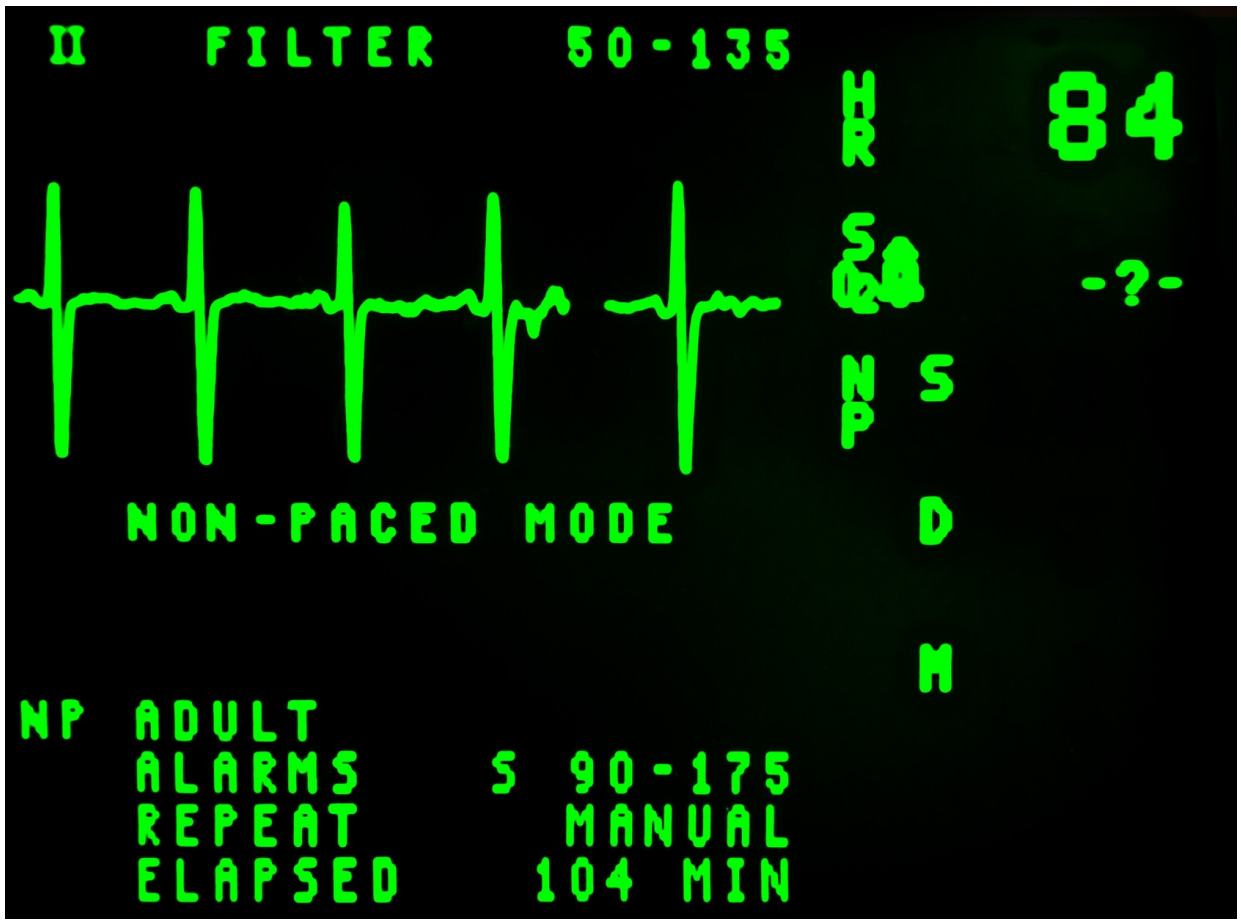


Image from Articulate Rise 360 Content Library.

Continue

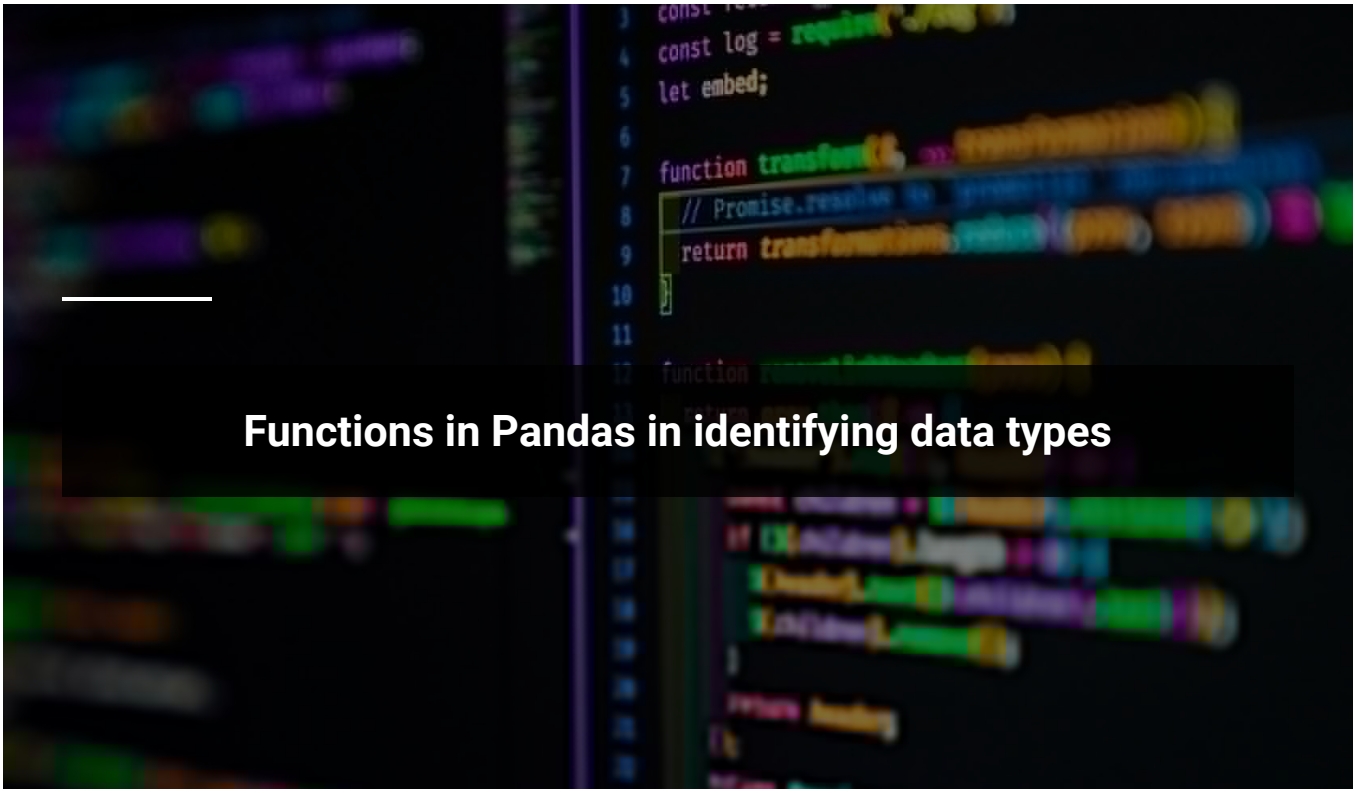


Image from Articulate Rise 360 Content Library.

Now that we are familiar with the data type and the kind of numerical data available, we will need practical knowledge to identify and retrieve the data type we are dealing with using Python Jupyter Notebook. You are most welcome to create a Jupyter Notebook and follow along with the examples presented in this section. In Pandas, we can use various functions to identify the attribute type within a dataframe. Note that we discussed in Lesson 1 that a dataframe can accommodate different data types within it. In this practical section, we will be looking at:

- Retrieve the data type for the whole dataframe
- Query about the data type for a specific attribute
- Retrieve attribute names with a certain data type

First, let us explore the dataset about customers we will be using in this section:

Age	Owned House	Salary (£)	Marital Status	Distance to the nearest town (km)
66	yes	54000	widowed	5.4
32	yes	321250	married	10.2
27	no	29120	married	6.7
40	no	46000	single	5.7
51	yes	61000	divorced	15
22	yes	52000	single	26

Figure 2.02. Customer dataset by Arden University. (Accessible code).

Now, for us to load the dataset onto a Jupyter Notebook, we can either:

- Load the dataset via a CSV file using Pandas, as shown in Lesson 1; or
- Create the dataset within the Jupyter Notebook.

In this example, we will use the second option as this will allow us to understand better how the syntax within Python may influence the data type.

The following is the Notebook where we created a dataframe using the data in Figure 2.02 above:

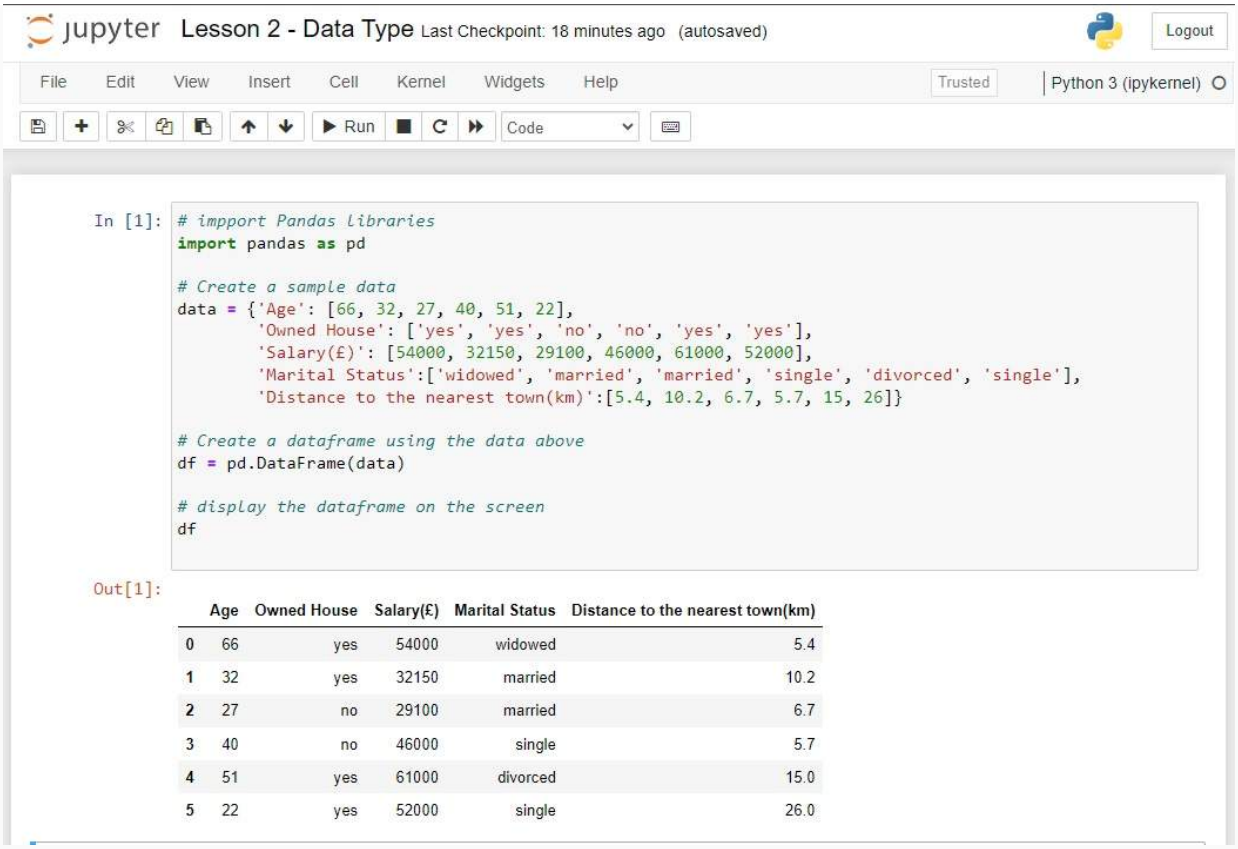


Figure 2.03. Customer dataset notebook in Jupyter Notebook by Arden University. (Accessible code).

Let us first retrieve all the data types for the dataframe. This can be accomplished by using the ***.dtypes*** method to retrieve the data type of all attributes. We can do this by adding the method to the Notebook above.

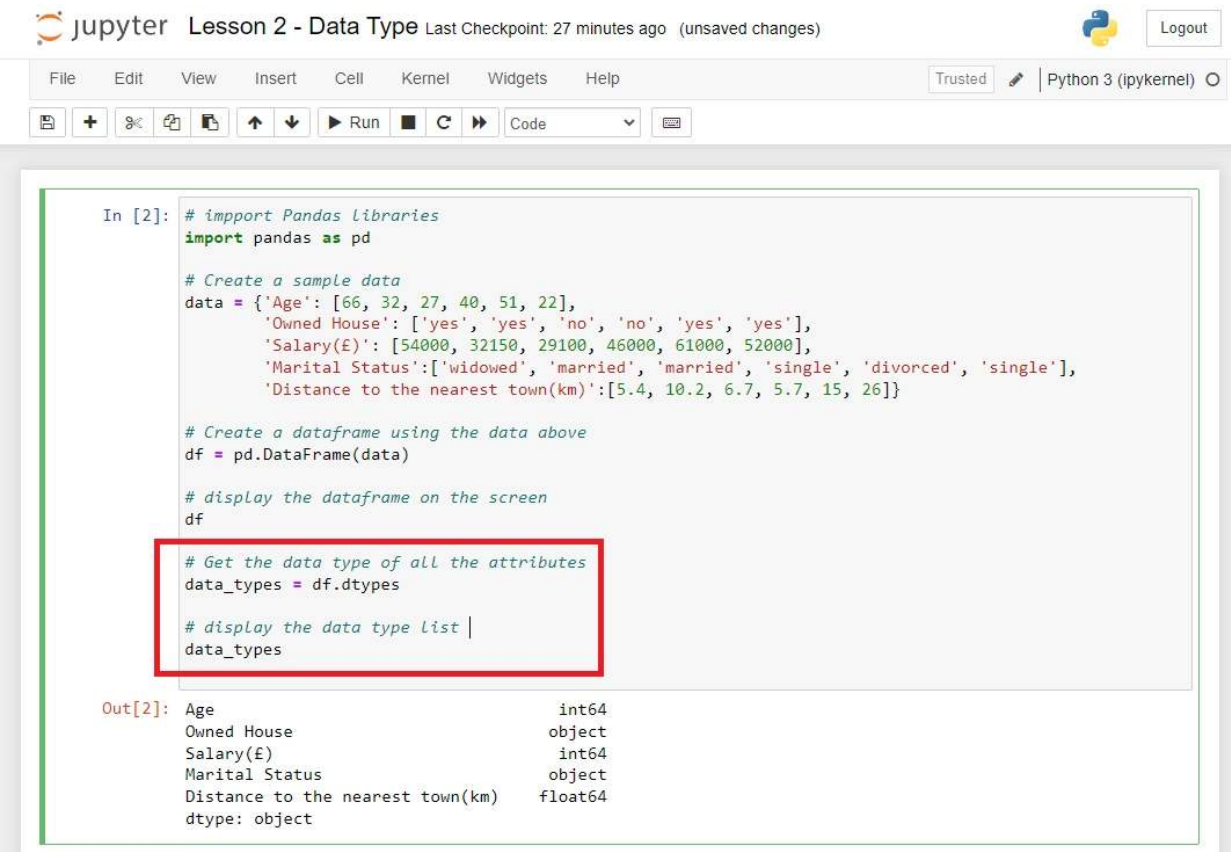


Figure 2.04. Customers' data type in Jupyter Notebook by Arden University. (Accessible code).

Continue

To interpret the data type outputs by Python, let us examine the following output table:

Attribute	Python output type	Data Type
Age	int64	Discrete numerical data

Attribute	Python output type	Data Type
Owned House	object	Nominal data
Salary (£)	int64	Discrete numerical data
Marital Status	object	Nominal data
Distance to the nearest town (km)	float64	Continuous data

Figure 2.05. Customers' data type output table by Arden University.

As we can see, we need to have fundamental knowledge of data types to identify the data types we are dealing with. However, the ***.dtypes*** function will provide us with the initial type for us to examine and, in essence, it provides us with the categories of the data, whether qualitative or quantitative.

If we want to retrieve the data type of a specific attribute, we can use the ***.dtypes*** on an attribute:

```
In [4]: # Check the data type of a specific column
        Age_data_type = df['Age'].dtype
        print(Age_data_type)

        int64
```

Figure 2.06. Customers' data attribute type in Jupyter Notebook by Arden University. (Accessible code).

In this case, we are retrieving the data type of **Age**, and the instructions will return *int64*.

Finally, we can also retrieve attributes of a particular type. For example, we can use the ***.select_dtypes()*** method to select only attributes with nominal and continuous data.

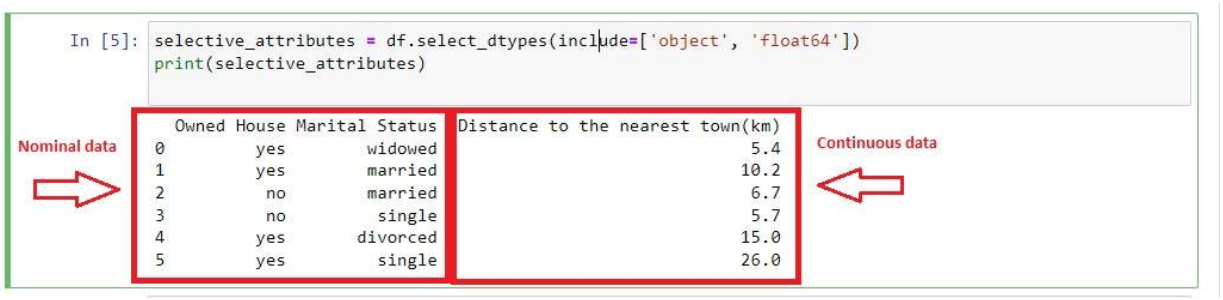


Figure 2.07. Select type in Jupyter Notebook by Arden University. (Accessible code).

Continue

Now, let us explore a bit more the syntax when we create our dataframe in our Notebook:

```
In [3]: # impport Pandas libraries
        import pandas as pd

        # Create a sample data
        data = {'Age': [66, 32, 27, 40, 51, 22],
                'Owned House': ['yes', 'yes', 'no', 'no', 'yes', 'yes'],
                'Salary(£)': [54000, 32150, 29100, 46000, 61000, 52000],
                'Marital Status':['widowed', 'married', 'married', 'single', 'divorced
                'Distance to the nearest town(km)':[5.4, 10.2, 6.7, 5.7, 15, 26]}

        # Create a dataframe using the data above
        df = pd.DataFrame(data)

        # display the dataframe on the screen
        df
```

Figure 2.08. Customer dataframe in Jupyter Notebook by Arden University. (Accessible code).

We can notice that when we declare an attribute as an object – in this case, **Owned House** and **Marital Status** – we put a single quote on the values to declare them as nominal data. What will happen if we put the same single quote on **Age**? Let us try.

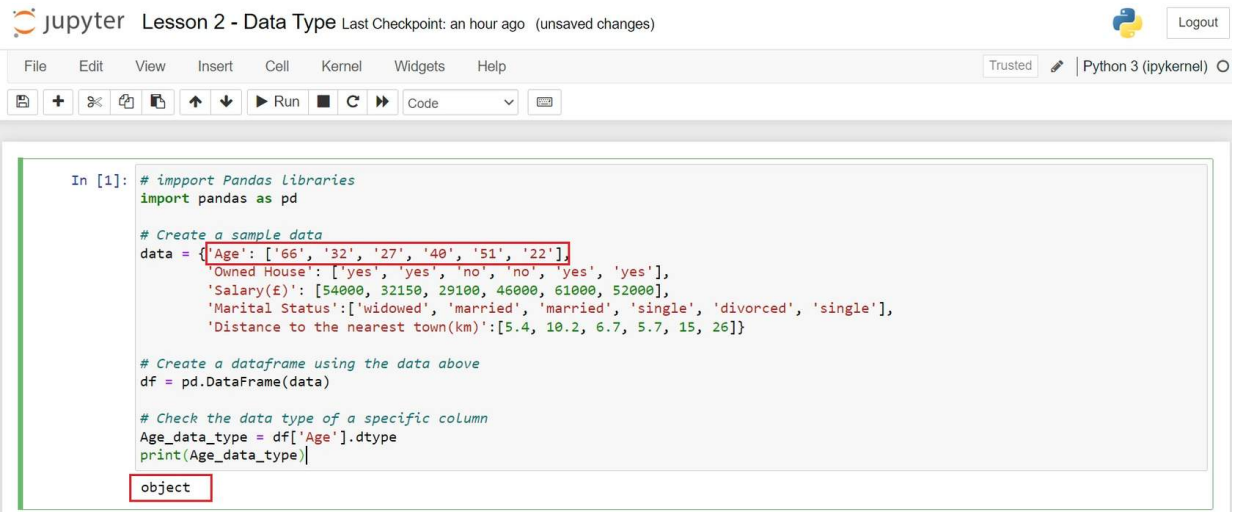


Figure 2.09. Customer dataframe practical in Jupyter Notebook by Arden University. (Accessible code).

As we see from the above, when we change an attribute value's syntax, the attribute's data type changes. In this example, **Age** has changed from discrete numerical data into nominal data, where the value of the number has no meaning but just a label.

Hence, one of the important takeaways from this topic is that when we examine our data collection and the data type, we must closely examine if the data types are appropriate for our purposes and if further processes need to be carried out. This brings this topic to a close, and we will explore the possible processes for data preparation in the next few topics.

Continue

Topic 1 - Activity 1

Data repositories

As part of becoming a data scientist, you must familiarise yourself with different datasets, from sales and marketing, to medical and scientific datasets, to datasets from the social economy domains. One way of doing this is to gain access to various open-source data repositories available to aspiring data scientists like us.

Explore the datasets from one of the following resources and examine the data type representation in the attributes.

The following are some of the resources:

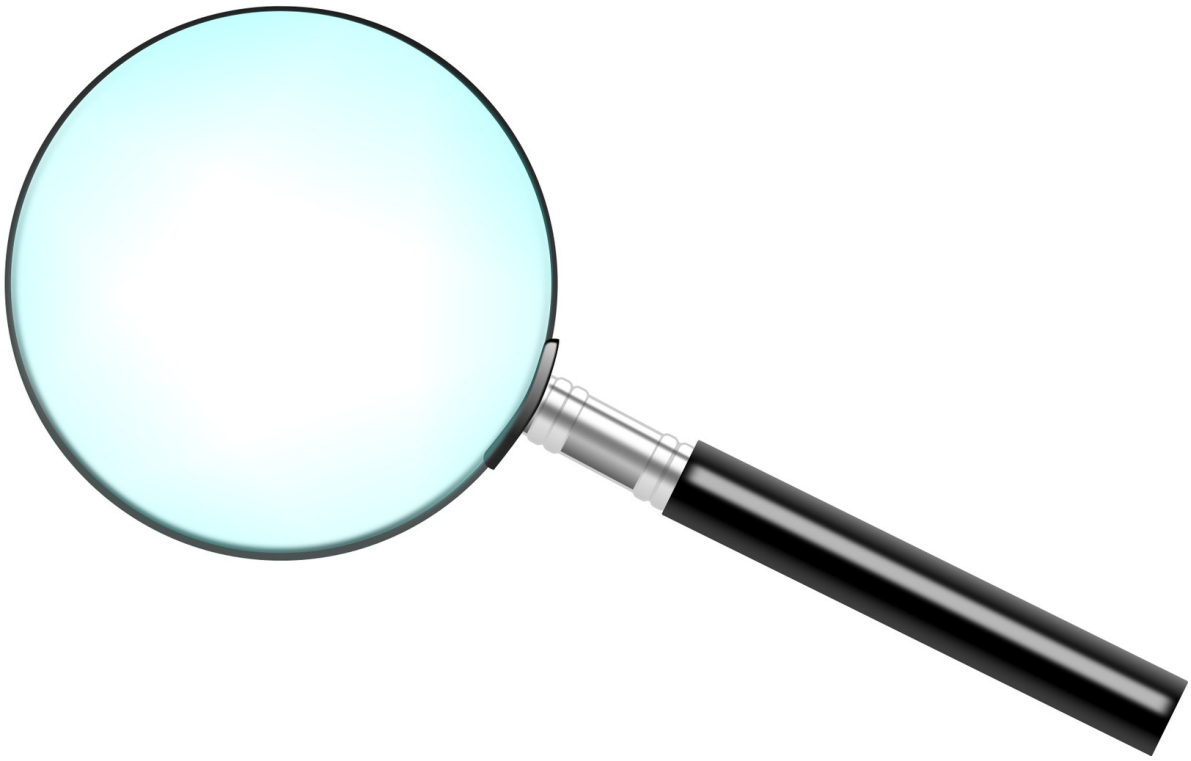


Image from Articulate Rise 360 Content Library.

---

1. UCI Machine Learning Repository

   This repository currently maintains about 657 datasets donated by the machine-learning community.

2. Kaggle

   This is one of the main resources for the AI and machine learning community to share, explore and test various algorithms using the 200,000+ datasets available.

3. HM Government

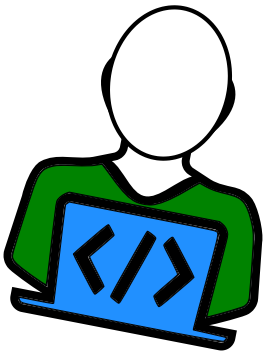
   This website provides some official datasets collected by various government agencies for the public.

4. KDnuggets

   This resource website provides access to various data repositories from leading global sources.

Continue

Topic 1 - Activity 2



Using the practical example in the topic as a guide, practise using the ***.dtypes*** method to retrieve the data type of the dataset you identified above. You may use 10 instances from the selected dataset.

Continue

Topic 1 - Activity 3

Reading

Pages 21-29 of Chapter 1: Introduction from Introduction to Data Mining, Global Edition by Tan *et al.*

Continue

Topic Summary

In this first topic of Lesson 2, we have been exposed to and examined the data types we will deal with as part of our development and life-long journey as data scientists. We have looked at the following data types:

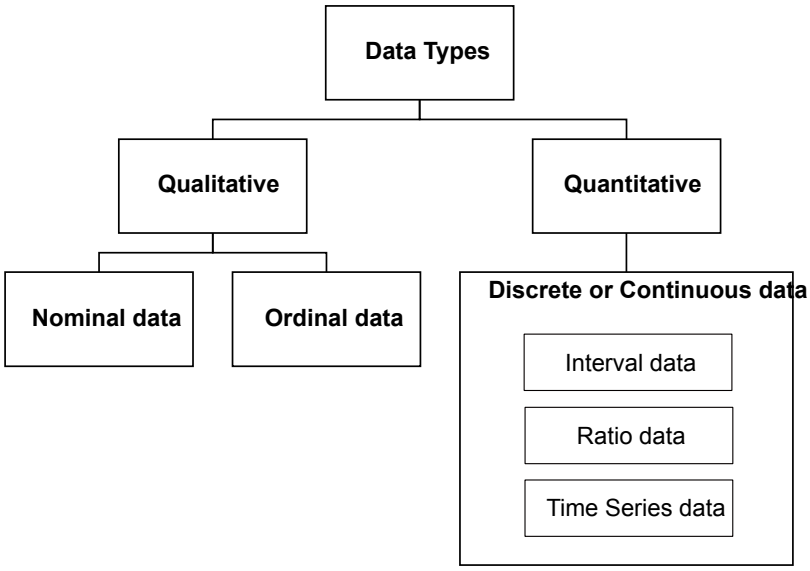


Figure 2.10. Customer dataframe practical by Arden University. (Accessible list).

Understanding the data type will allow us to utilise the data better when implementing the most appropriate machine-learning algorithms. With this knowledge, we furthered our practical skills by learning and practising using the Pandas functions ***.dtype***. The activities provided opportunities to enhance your understanding of data types and exposure to various data resources you will use for the module assessment or your further development. Remember to update your Python cheat sheet with this function.

Now that we understand the data types, we can improve our data quality by cleaning them.

Continue to Topic 2

Topic 2: Data cleaning

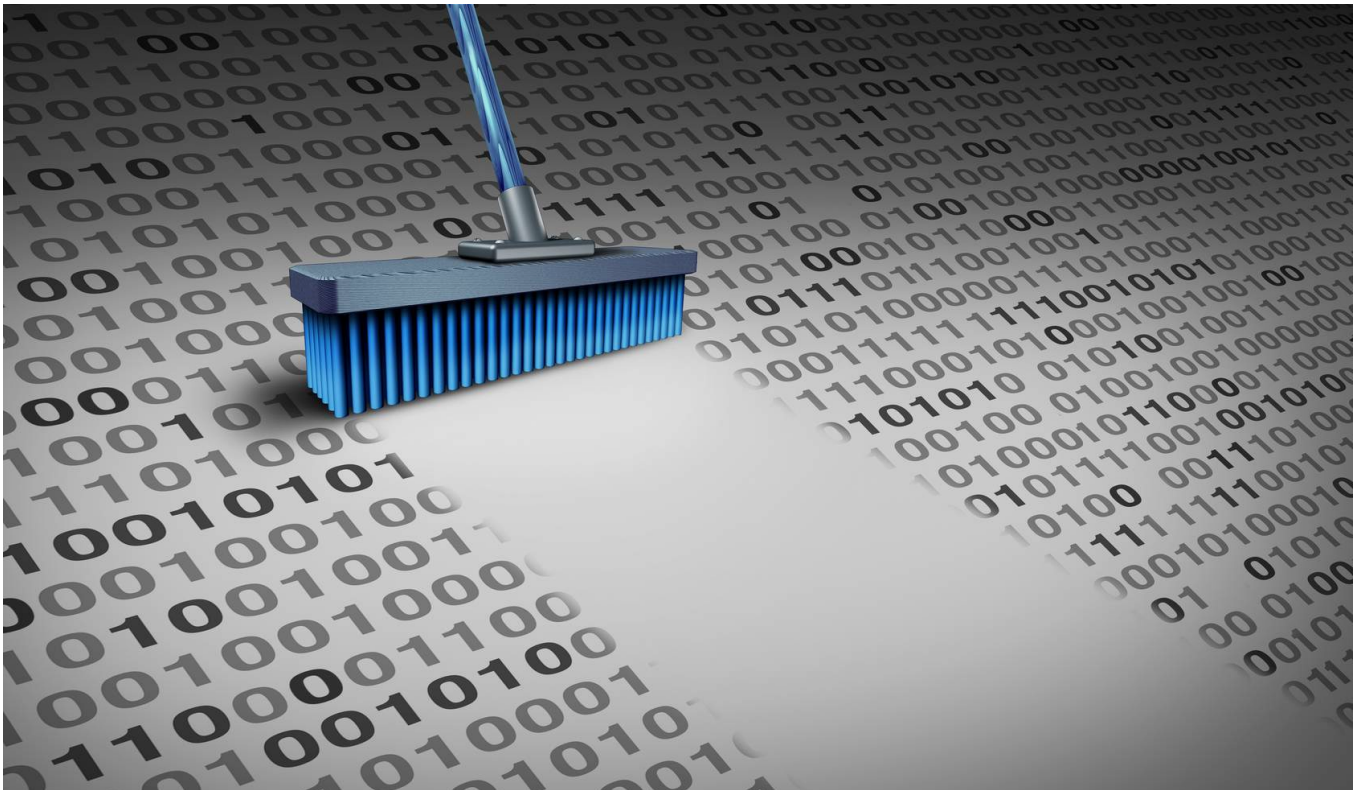


Image by Lightspring from Shutterstock.com.

Did you know?

LinkedIn lists hundreds of thousands of jobs requiring machine learning skills worldwide (LinkedIn, 2023).

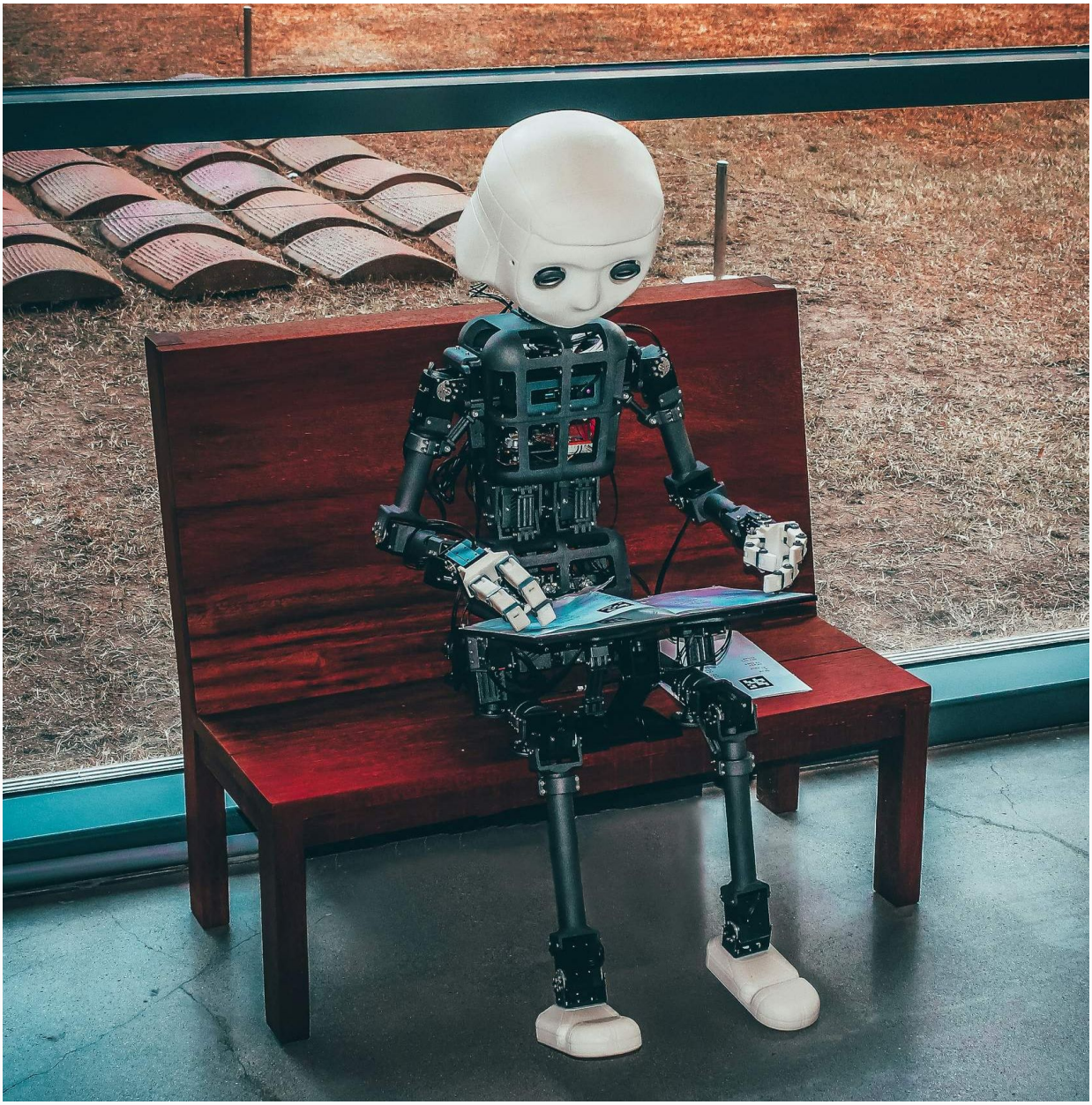


Image from Articulate Rise 360 Content Library.

Continue

In 2020, what do you think the main reason for investing in machine learning was?

- To increase production speed
- To profile customers
- To reduce company costs
- To generate new products

SUBMIT

Continue

Image from Articulate Rise 360 Content Library.

In 2020, the main purpose of investing in machine learning was to reduce company costs (38%) (BasuMallick, 2021).

Continue

The data collection process comes with various obstacles. Quite often, the dataset presented comes with:

- Missing data due to machine malfunctions when recording a reading
- Duplicate data recorded

- A customer did not answer certain questions in a customer survey
- Wrong unit when data sources come from various databases (e.g., data sources in the US recording distance in kilometres, while data sources in the UK record it in miles)
- Wrong data type when processing is carried out (e.g., age recorded as an object (nominal) data type).

In this topic, we will demonstrate several practical techniques to overcome these issues. Let's start off with resolving missing data.

Continue

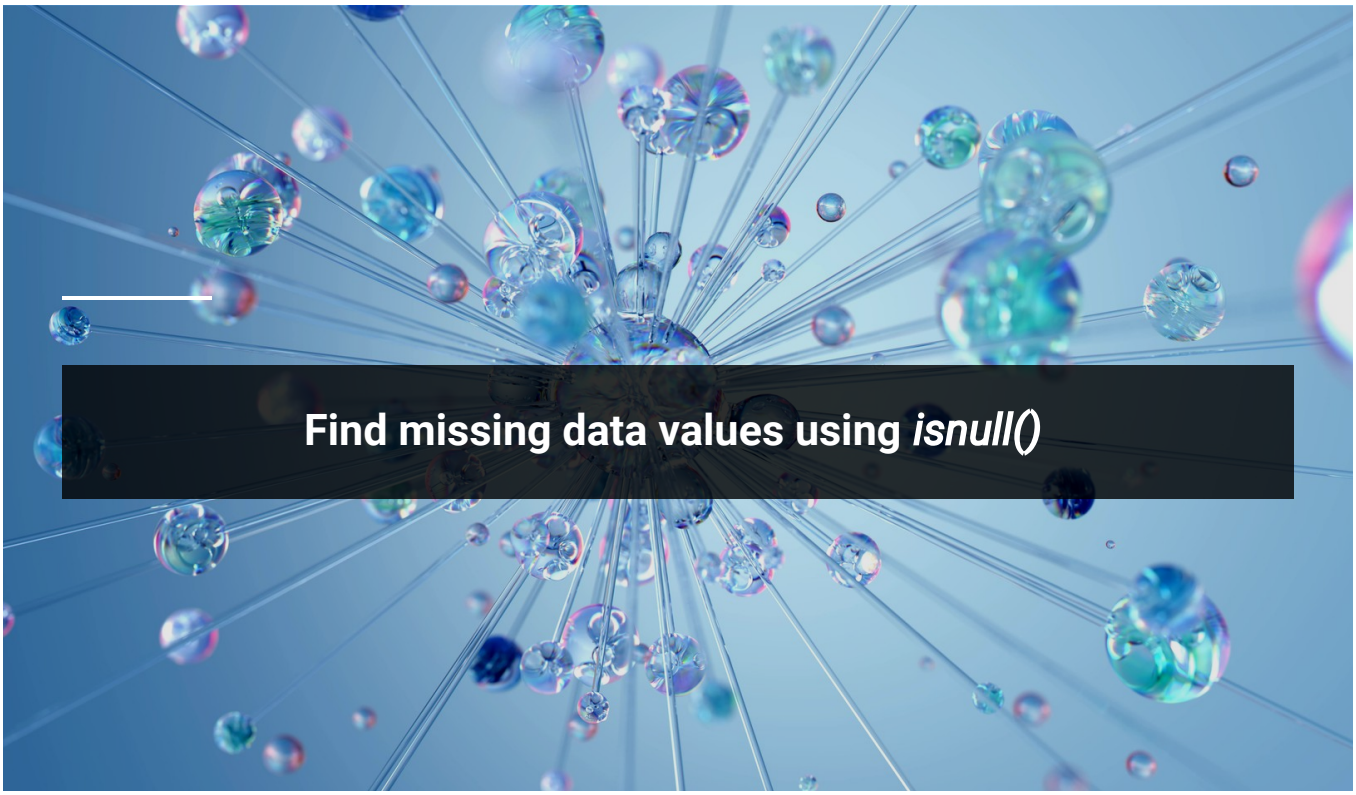
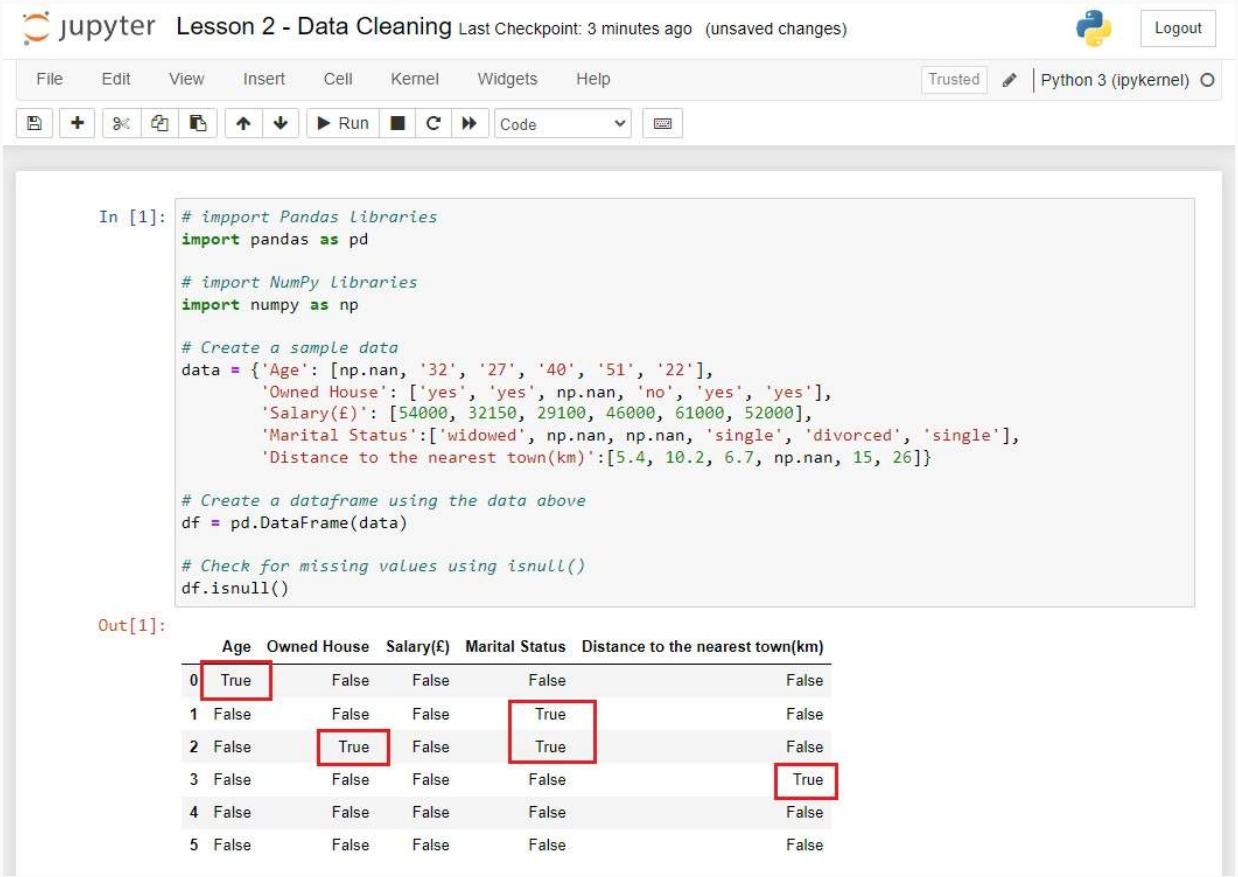


Image from Articulate Rise 360 Content Library.

Let us start by finding if missing values exist in the dataset by returning True for a missing value and False if data is present in the data instances. In this example, we will use a specific NumPy function called NaN to represent missing values in a dataset.

Example 1: Finding missing data with Boolean results

In this example, we will use the same dataset we have covered in Topic 1 of the lesson. In this case, we have purposely included some missing data for demonstration purposes.




```
In [1]: # impport Pandas libraries
import pandas as pd

# import NumPy libraries
import numpy as np

# Create a sample data
data = {'Age': [np.nan, '32', '27', '40', '51', '22'],
        'Owned House': ['yes', 'yes', np.nan, 'no', 'yes', 'yes'],
        'Salary(£)': [54000, 32150, 29100, 46000, 61000, 52000],
        'Marital Status':['widowed', np.nan, np.nan, 'single', 'divorced', 'single'],
        'Distance to the nearest town(km)':[5.4, 10.2, 6.7, np.nan, 15, 26]}

# Create a dataframe using the data above
df = pd.DataFrame(data)

# Check for missing values using isnull()
df.isnull()
```

Out[1]:

	Age	Owned House	Salary(£)	Marital Status	Distance to the nearest town(km)
0	True	False	False	False	False
1	False	False	False	True	False
2	False	True	False	True	False
3	False	False	False	False	True
4	False	False	False	False	False
5	False	False	False	False	False

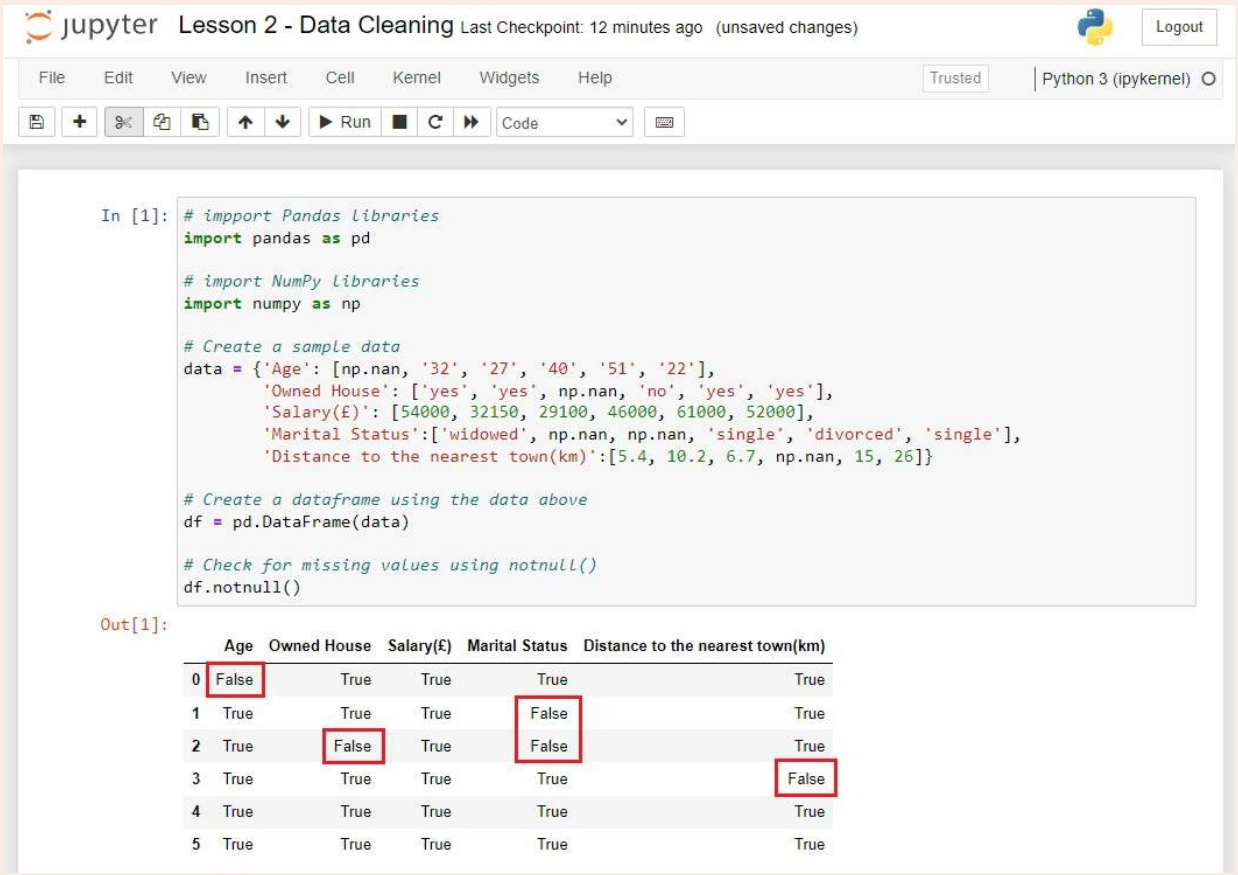
Figure 2.11. Finding missing values in Jupyter Notebook by Arden University. (Accessible code).

As we can see from the outputs above, any missing values will return *True* when detected.

Continue

Example 2: Using *notnull()* instead to detect if missing values are not present

In this example, the ***notnull()*** will return False for missing values.




```
In [1]: # impport Pandas libraries
import pandas as pd

# import NumPy libraries
import numpy as np

# Create a sample data
data = {'Age': [np.nan, '32', '27', '40', '51', '22'],
        'Owned House': ['yes', 'yes', np.nan, 'no', 'yes', 'yes'],
        'Salary(£)': [54000, 32150, 29100, 46000, 61000, 52000],
        'Marital Status':['widowed', np.nan, np.nan, 'single', 'divorced', 'single'],
        'Distance to the nearest town(km)':[5.4, 10.2, 6.7, np.nan, 15, 26]}

# Create a dataframe using the data above
df = pd.DataFrame(data)

# Check for missing values using notnull()
df.notnull()
```

Out[1]:

	Age	Owned House	Salary(£)	Marital Status	Distance to the nearest town(km)
0	False	True	True	True	True
1	True	True	True	False	True
2	True	False	True	False	True
3	True	True	True	True	False
4	True	True	True	True	True
5	True	True	True	True	True

Figure 2.12. Finding non-missing values in Jupyter Notebook by Arden University. (Accessible code).

We might be thinking: Which function should we use? The answer is either, as it is based on your preference. This is the beauty of Python: you can use any functions available to achieve the same results.

Continue

Example 3: Replace missing or null values using replace() method

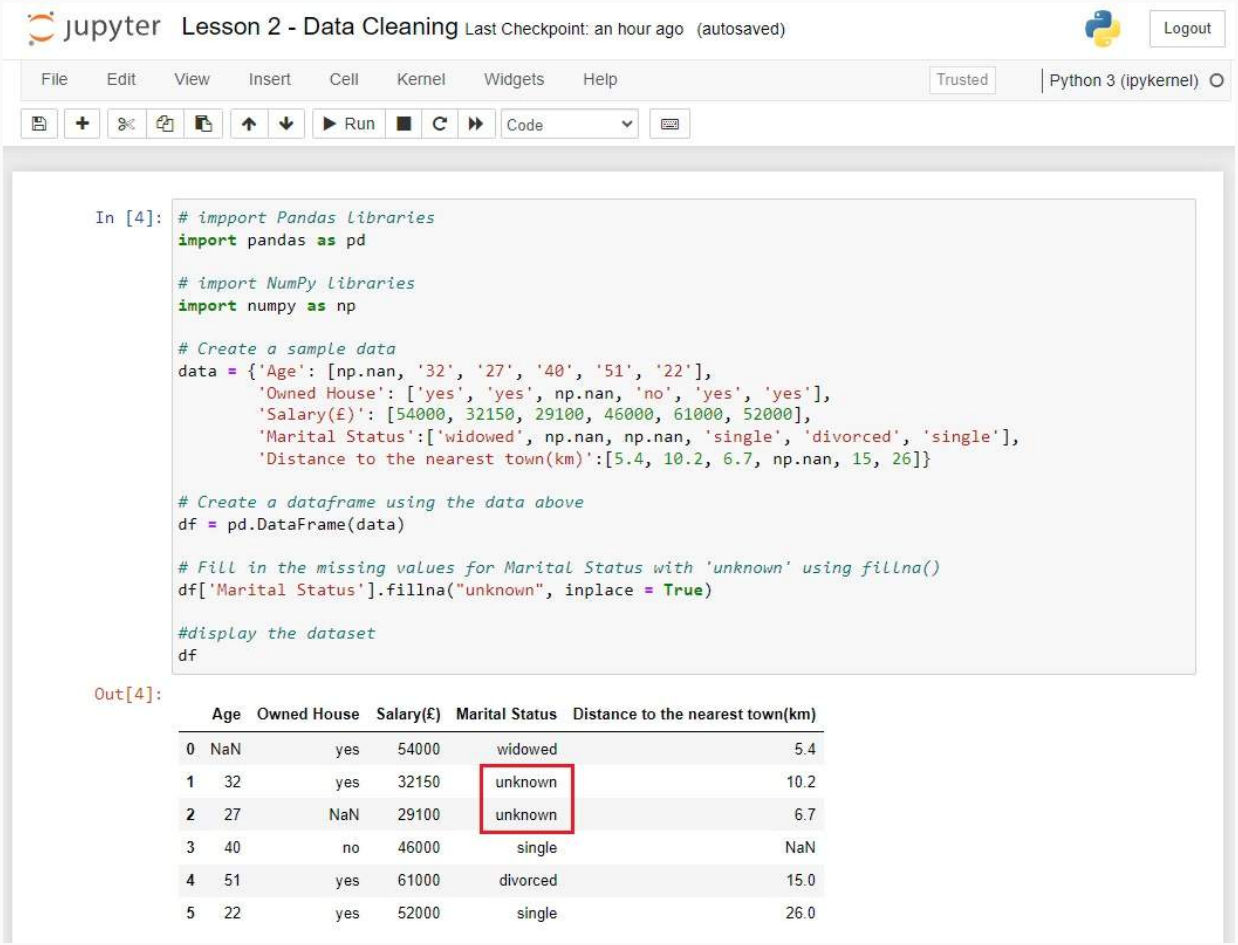
Now that we know that a null value exists in our dataset, we can use the ***replace()*** method. We might wonder what value we can use to replace a null value. Now, this is an art. We must study the dataset and decide on a replacement that will suit our purposes. This might be:

- Replace with the previous data value
- Replace with the mean value if it is a numerical attribute
- Replace with a distinct value or label

Why do we replace null values within a data set with added values that may not represent or be consistent with all other data points within that identified variable? Replacing missing values in the most unbiased way possible with a consistent value / label can improve the overall results of the study. But why are the values missing in the first place? The answer to this can be multiple reasons, such as questions maybe too sensitive for a certain age or ethnicity group to answer. Participants may also be reluctant to provide income details, or certain medical conditions. By using consistent replacement values can therefore mitigate the gaps in the data set.

In other scenarios, entire data rows / columns can be removed altogether and below we will demonstrate how this can be achieved through various commands. Caution must be adhered to however when dropping rows and columns, as this can and will affect the overall investigation.

Let us look at the example. We can replace the null value of **Marital Status** with *unknown*.




```
In [4]: # impport Pandas libraries
        import pandas as pd

        # import NumPy libraries
        import numpy as np

        # Create a sample data
        data = {'Age': [np.nan, '32', '27', '40', '51', '22'],
                'Owned House': ['yes', 'yes', np.nan, 'no', 'yes', 'yes'],
                'Salary(£)': [54000, 32150, 29100, 46000, 61000, 52000],
                'Marital Status':['widowed', np.nan, np.nan, 'single', 'divorced', 'single'],
                'Distance to the nearest town(km)':[5.4, 10.2, 6.7, np.nan, 15, 26]}

        # Create a dataframe using the data above
        df = pd.DataFrame(data)

        # Fill in the missing values for Marital Status with 'unknown' using fillna()
        df['Marital Status'].fillna("unknown", inplace = True)

        #display the dataset
        df
```

Out[4]:

	Age	Owned House	Salary(£)	Marital Status	Distance to the nearest town(km)
0	NaN	yes	54000	widowed	5.4
1	32	yes	32150	unknown	10.2
2	27	NaN	29100	unknown	6.7
3	40	no	46000	single	NaN
4	51	yes	61000	divorced	15.0
5	22	yes	52000	single	26.0

Figure 2.13. Missing values replacement in Jupyter Notebook by Arden University. (Accessible code).

Continue

Example 4: Drop the instance if replacement is impossible using *dropna()*

Let's say we have an instance or several instances in a large dataset that have several missing values, and it is impossible to do any replacement due to a lack of information in the instance. Removing the instance with more than three NaN will be better in this case. For example:

Age	Owned House	Salary (£)	Marital Status	Distance to the nearest town (km)
NaN	NaN	NaN	widowed	NaN
32	yes	32150	Unknown	10.2
27	NaN	29120	Unknown	6.7
40	no	46000	single	NaN
51	yes	61000	divorced	15
22	yes	52000	single	26

Figure 2.14. Missing values removal by Arden University. (Accessible code).

The implementation in the Notebook is as follows:

jupyter Lesson 2 - Data Cleaning Last Checkpoint: an hour ago (autosaved)

Logout

File Edit View Insert Cell Kernel Widgets Help Trusted Python 3 (ipykernel)

Run Code

In [1]:
```
# impport Pandas libraries
import pandas as pd

# import NumPy libraries
import numpy as np

# Create a sample data
data = {'Age': [np.nan, '32', '27', '40', '51', '22'],
        'Owned House': [np.nan, 'yes', np.nan, 'no', 'yes', 'yes'],
        'Salary(£)': [np.nan, 32150, 29100, 46000, 61000, 52000],
        'Marital Status':['widowed', 'unknown', 'unknown', 'single', 'divorced', 'single'],
        'Distance to the nearest town(km)':[np.nan, 10.2, 6.7, np.nan, 15, 26]}

# Create a dataframe using the data above
df = pd.DataFrame(data)

# display the dataframe
df
```

Out[1]:

	Age	Owned House	Salary(£)	Marital Status	Distance to the nearest town(km)	
0	NaN	NaN	NaN	widowed	NaN	Instance we want to remove
1	32	yes	32150.0	unknown	10.2	
2	27	NaN	29100.0	unknown	6.7	
3	40	no	46000.0	single	NaN	
4	51	yes	61000.0	divorced	15.0	
5	22	yes	52000.0	single	26.0	

Figure 2.15. Missing values removal notebook in Jupyter Notebook by Arden University. (Accessible code).

The removal of the instance can be done using the following:

In [2]:
```
# Set the threshold of the number NaN to 3 for the instaces we want to remove
df_new = df.dropna(thresh = 3)

# display the cleaned dataframe
df_new
```

Out[2]:

	Age	Owned House	Salary(£)	Marital Status	Distance to the nearest town(km)
1	32	yes	32150.0	unknown	10.2
2	27	NaN	29100.0	unknown	6.7
3	40	no	46000.0	single	NaN
4	51	yes	61000.0	divorced	15.0
5	22	yes	52000.0	single	26.0

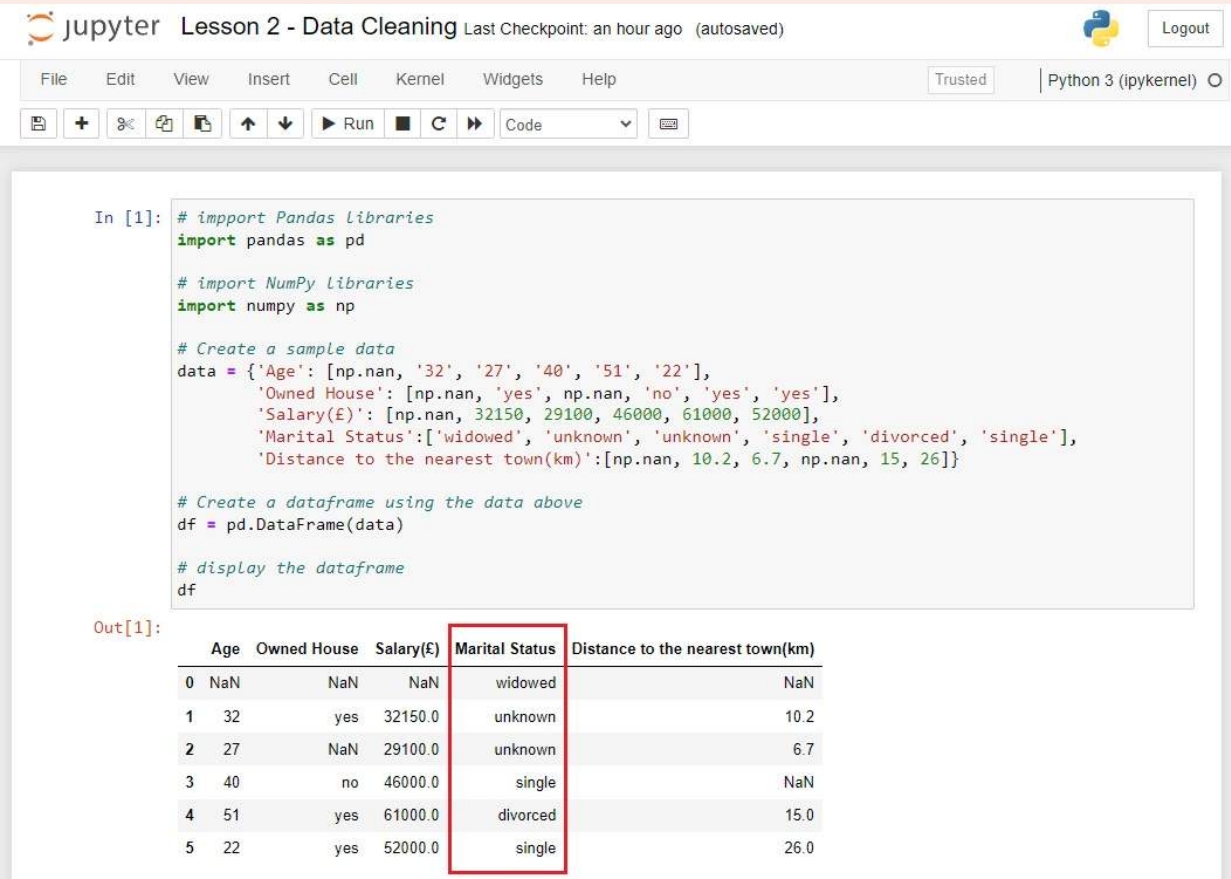
Figure 2.16. Missing values removal implementation in Jupyter Notebook by Arden University. (Accessible code).

As you can see, the first instance has been removed.

Continue

Example 5: Drop the attribute if replacement is impossible using *drop()*

What if we want to remove an attribute with many missing values? Let's say we want to drop the **Marital Status** attribute from our dataset:



	Age	Owned House	Salary(£)	Marital Status	Distance to the nearest town(km)
0	NaN	NaN	NaN	widowed	NaN
1	32	yes	32150.0	unknown	10.2
2	27	NaN	29100.0	unknown	6.7
3	40	no	46000.0	single	NaN
4	51	yes	61000.0	divorced	15.0
5	22	yes	52000.0	single	26.0

Figure 2.17. Attribute removal in Jupyter Notebook by Arden University. (Accessible code).

After running the code above, we are ready to drop the **Marital Status** attribute:

```
In [2]: # Drop the attribute 'Marital Status', the axis we want to drop is a columns or 1
        df_new = df.drop('Marital Status', axis = 'columns')

        # display the cleaned dataframe
        df_new
```

Out[2]:

	Age	Owned House	Salary(£)	Distance to the nearest town(km)
0	NaN	NaN	NaN	NaN
1	32	yes	32150.0	10.2
2	27	NaN	29100.0	6.7
3	40	no	46000.0	NaN
4	51	yes	61000.0	15.0
5	22	yes	52000.0	26.0

Figure 2.18. Attribute removal implementation in Jupyter Notebook by Arden University. (Accessible code).

As you can see, the **Marital Status** attribute has been removed from the dataset.

Continue

Example 6: Change the data type using *.astype()*

At the end of Topic 1, we discussed the possibility of error in the data type we use for an attribute. This is shown again in the figure below:

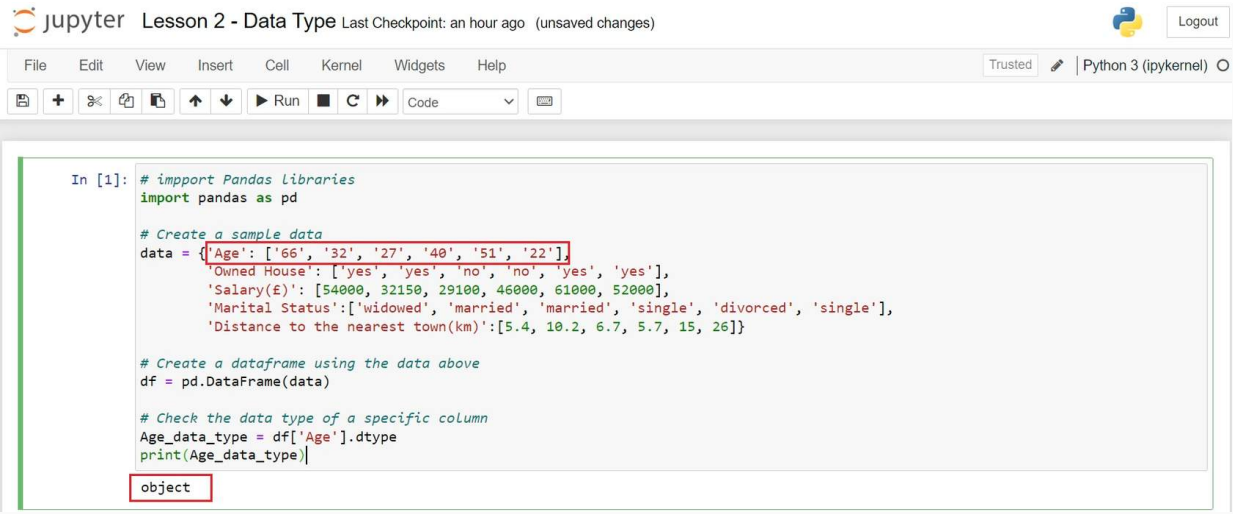


Figure 2.19a. Customer dataframe practical in Jupyter Notebook by Arden University. (Accessible code).

Now, we can change the data type of the attribute. For consistency, we will continue using the dataset we are currently using in this topic.

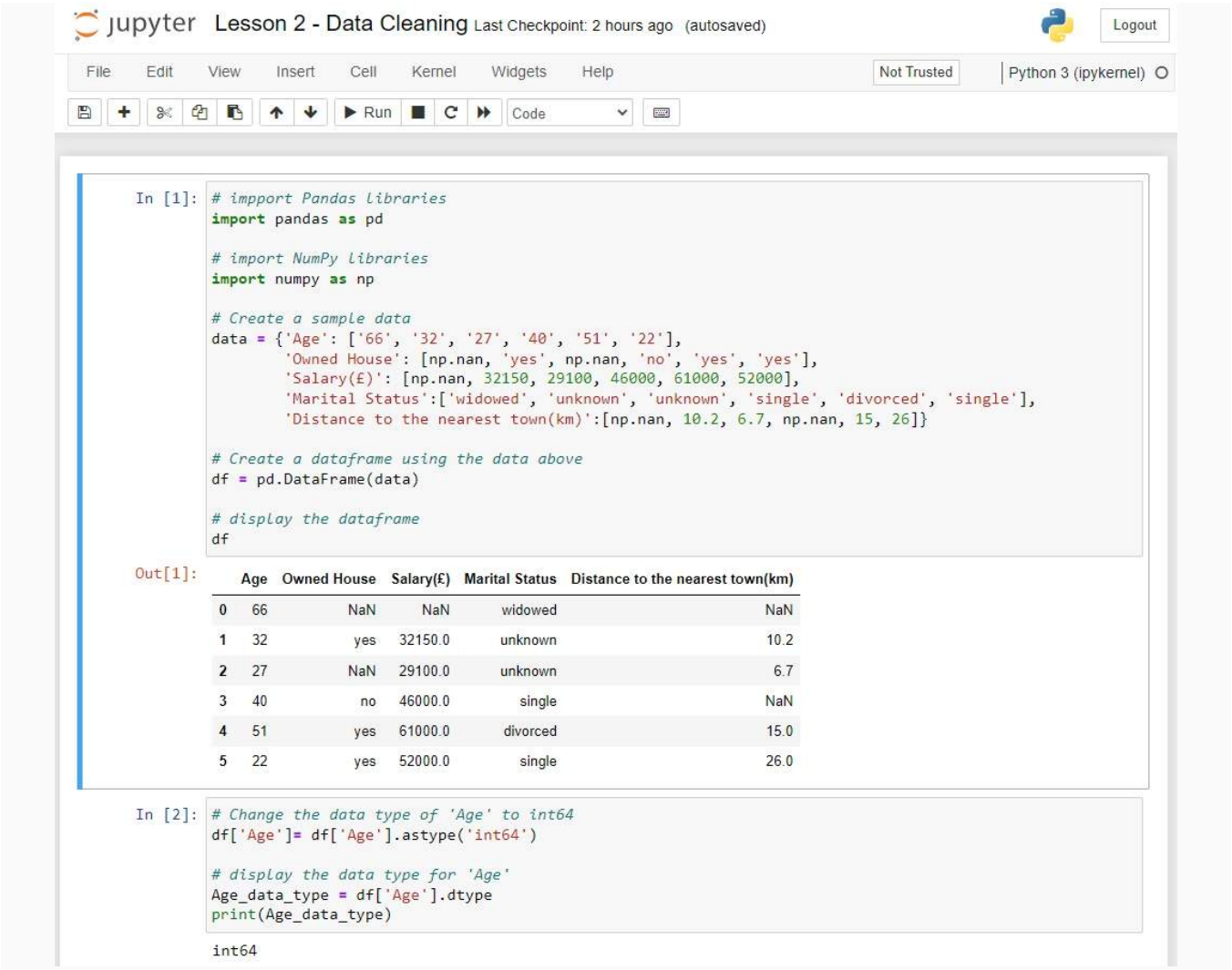


Figure 2.19b. Change attribute data type in Jupyter Notebook by Arden University. (Accessible code).

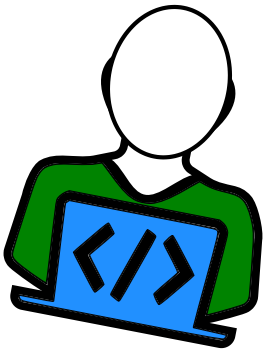
Continue

Topic 2 - Activity 1

Practical exercise using *replace()* method

Using **Example 3: Replace missing or null values using replace() method,** replace the null values in Distance to the nearest town(km) with the mean value of the attribute.

Hints: df['*column_name*'].fillna(df['*column_name*'].mean(), inplace=True)



Continue

Feedback

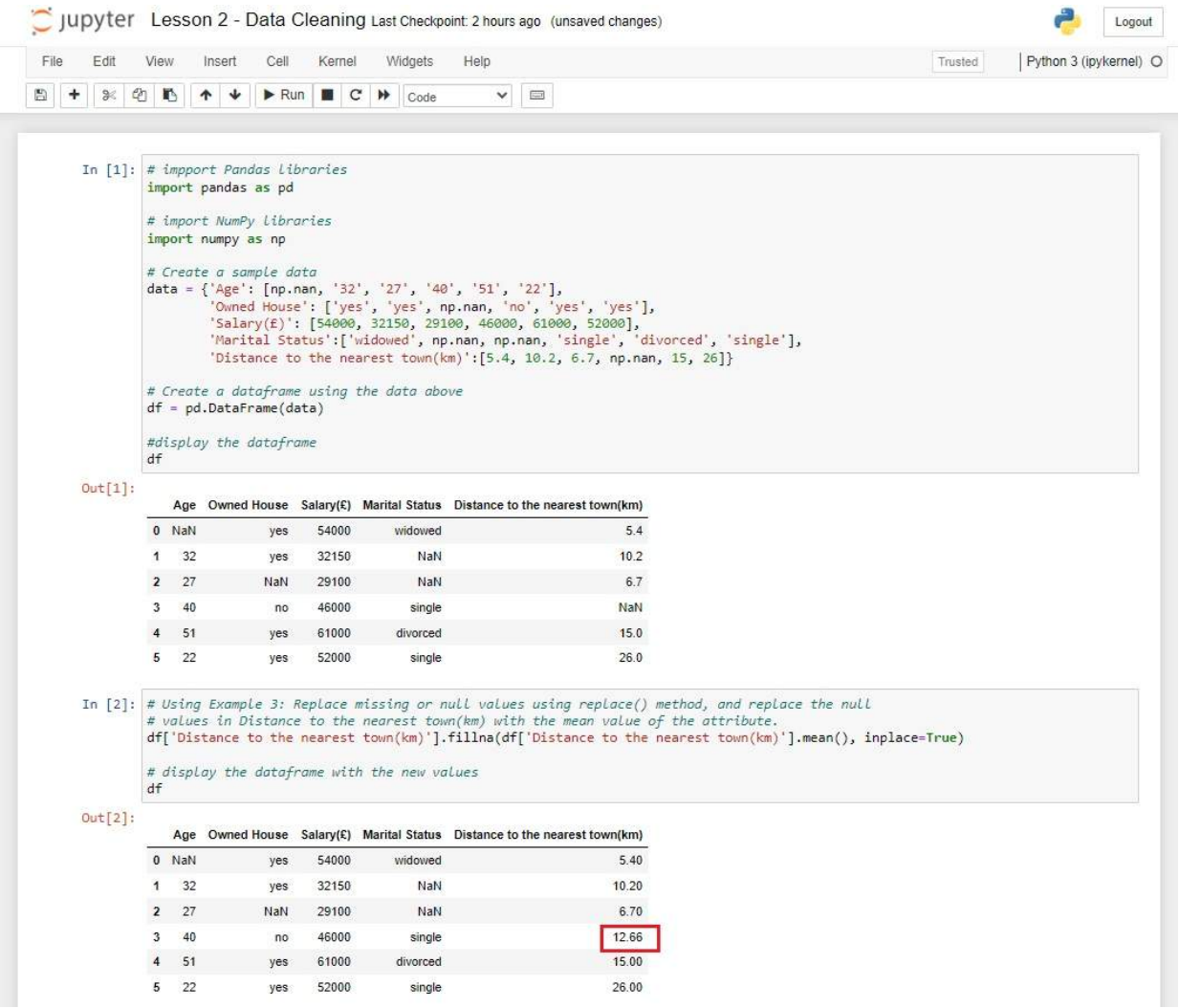
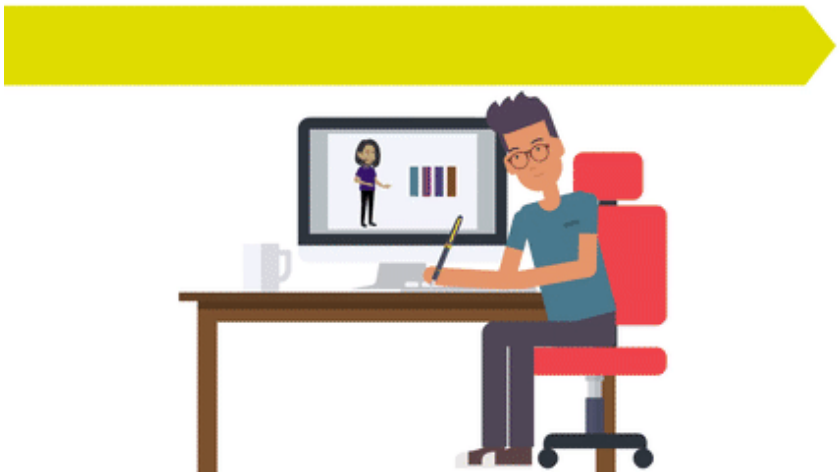


Figure 2.20. Practical exercise using replace() solution in Jupyter Notebook by Arden University. (Accessible code).

Continue

Topic 2 - Activity 2

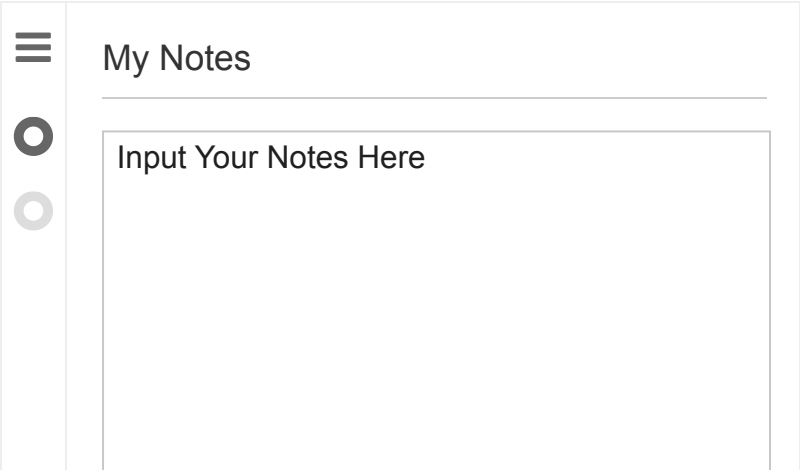
Make a Note



It is time to update your cheat sheet with all the functions you learned in this topic. Feel free to explore this topic further using any resources you can find online and add to your repertoire of functions for data cleaning.

You can use the below Notepad to help you with this activity, but feel free to use whichever note-taking tool you prefer.

If you decide to use the below Notepad, please remember anything you type in the box below will not be kept when you leave the lesson, so you will need to download your notes for revision and discussion purposes.



Continue

Topic 2 - Activity 3

Reading

Pages 20-30 of Chapter 2: Data Preprocessing from Data Mining and Predictive Analytics by Larose and Larose

Pages 22-28 of Part 2: Understanding and describing your data from How to Analyze Data by Radcliffe

Continue

TOPIC SUMMARY

In this topic, we have investigated various techniques to conduct data cleaning. In the real world, messy data is a real issue, and the first step as a data scientist is to ensure that the data we will use is consistent and has minimal errors. If we use "unclean" data, the analysis process can cause serious issues in gaining insights into the data, resulting in bad decision-making.

We have dived into various practical functions as summarised below:

- Finding missing data values using ***isnull()***
- Using missing or null values using ***replace()*** method
- Replacing missing or null values using ***replace()*** method
- Dropping the instance if replacement is impossible using ***dropna()***
- Dropping the attribute if replacement is impossible using ***drop()***
- Changing the data type using .***astype()***.

Now that we have the skills to clean our data, we are ready to proceed to the next topic of this lesson: data integration.

Continue to Topic 3

Topic 3: Data integration

Image by Omelchenko from Shutterstock.com.

Data integration is a key part of the work of a data engineer, and it is a role that a data scientist may take on. It is the process of merging data from various sources so that the final data can be sampled or extracted for analysis using machine-learning techniques. For example, medical records that support your home doctor may result from integrating various databases into a collective database.

This topic will explore some practical techniques used in this process. Before exploring the topic's practical elements, let us summarise the general tools and techniques in data integration.

MANUAL INTEGRATION | MIDDLEWARE INTEGRATION | COMMON TOOLS

Manual integration is a technique that does not require automation and can be appropriate if the database is small and limited. However, it is unsustainable with company growth.

MANUAL INTEGRATION | MIDDLEWARE INTEGRATION | COMMON TOOLS

This technique is used when data is taken from many different sources: the data are normalised and stored in a resulting dataset. The technique is commonly used when a company wishes to integrate data from legacy to modern systems.

MANUAL INTEGRATION | MIDDLEWARE INTEGRATION | COMMON TOOLS

On-premises data integration tool integrates data from local sources and connects legacy databases using the middleware technique mentioned earlier.

Open-source data integration tool will, wherever possible, avoid expensive enterprise solutions, and is usually cloud-based. However, the user will be solely responsible for the security and privacy of the data when using this tool.

Now that we understand the general tools and techniques used in the industry, let's dive into the practical techniques you can utilise when integrating various datasets.

Here are the datasets we will be using in this topic:

Age	Owned House	Salary (£)	Marital Status	Distance to the nearest town (km)
66	yes	54000	widowed	5.4
32	yes	321250	married	10.2
27	no	29120	married	6.7
40	no	46000	single	5.7
51	yes	61000	divorced	15
22	yes	52000	single	26

Figure 2.02. Customer dataset by Arden University. (Accessible code).

Gender	Salary (£)	Car Owner
Male	54000	no
female	32150	yes
Non-binary	29120	yes

Gender	Salary (£)	Car Owner
male	46000	no
female	61000	no
male	52000	yes

Figure 2.21. Additional customer dataset by Arden University.

The datasets can be created in the Notebook as follows:

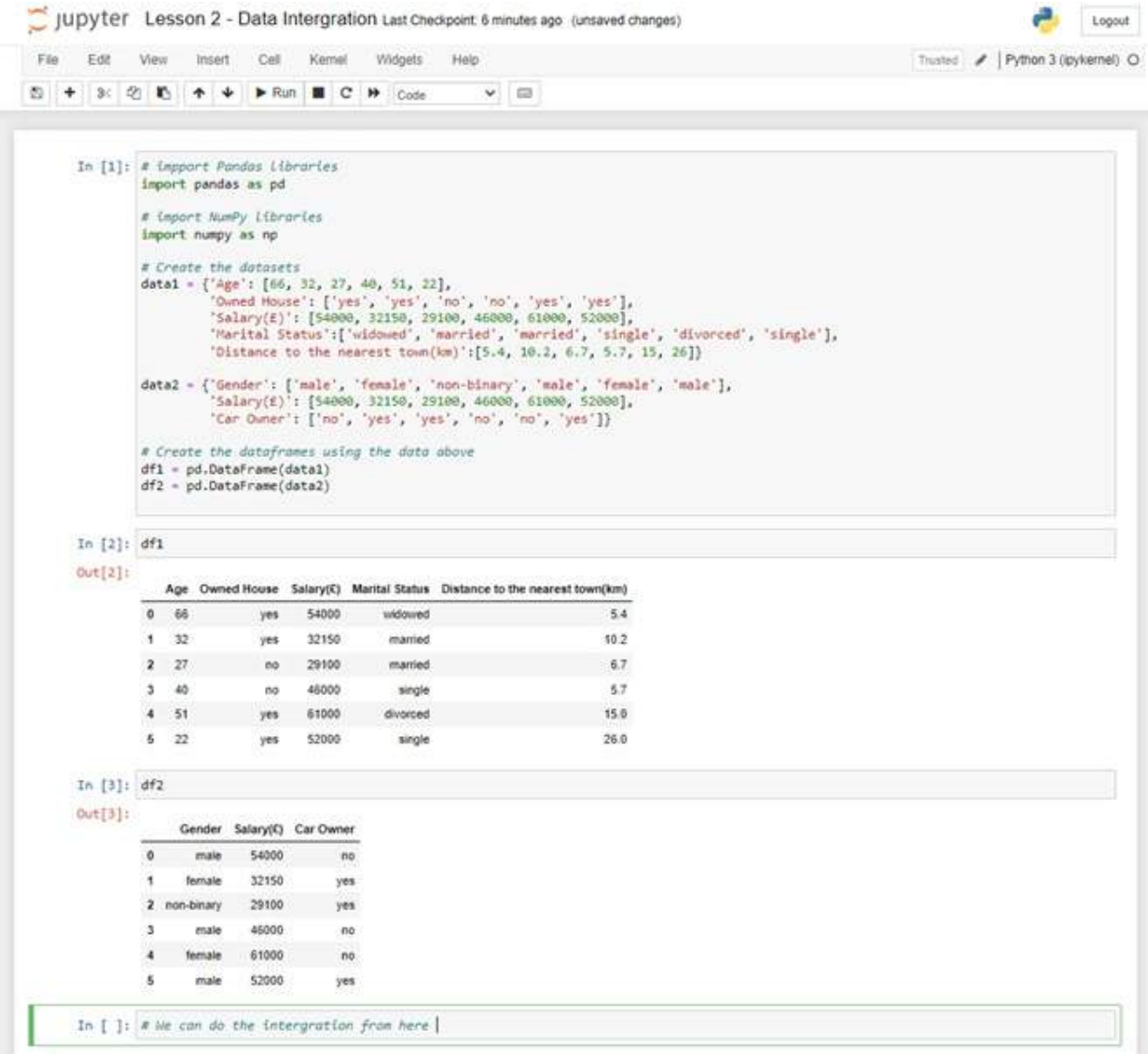


Figure 2.22. Customer datasets in Jupyter Notebook by Arden University. (Accessible code).

Continue

Example 1: Merging the two datasets using *pd.merge()*

We can first merge the two dataframes using a similar attribute as the key – in this case, the Salary attribute.

```
In [6]: # We can do the intergration from here
merged_df = pd.merge(df1, df2, on='Salary(£)', how='inner')

# display the merged dataframe
merged_df
```

Out[6]:

	Age	Owned House	Salary(£)	Marital Status	Distance to the nearest town(km)	Gender	Car Owner
0	66	yes	54000	widowed	5.4	male	no
1	32	yes	32150	married	10.2	female	yes
2	27	no	29100	married	6.7	non-binary	yes
3	40	no	46000	single	5.7	male	no
4	51	yes	61000	divorced	15.0	female	no
5	22	yes	52000	single	26.0	male	yes

Figure 2.23. Customer datasets merge in Jupyter Notebook by Arden University. (Accessible code).

As we can see from the output, the two dataframes are merged using salary as key.

Continue

Example 2: Concatenating the two datasets using *pd.concat()*

In this example, we will concatenate the two dataframes vertically (axis = 0) and horizontally (axis = 1).

Let's start by adding the dataframe vertically:

```
In [10]: # We can do the intergration from here
concatenated_df = pd.concat([df1, df2], axis=0)  # Concatenate along rows (vertically)

# display the concatenated dataframe
concatenated_df
```

Out[10]:

	Age	Owned House	Salary(£)	Marital Status	Distance to the nearest town(km)	Gender	Car Owner
0	66.0	yes	54000	widowed	5.4	NaN	NaN
1	32.0	yes	32150	married	10.2	NaN	NaN
2	27.0	no	29100	married	6.7	NaN	NaN
3	40.0	no	46000	single	5.7	NaN	NaN
4	51.0	yes	61000	divorced	15.0	NaN	NaN
5	22.0	yes	52000	single	26.0	NaN	NaN
0	NaN	NaN	54000	NaN	NaN	male	no
1	NaN	NaN	32150	NaN	NaN	female	yes
2	NaN	NaN	29100	NaN	NaN	non-binary	yes
3	NaN	NaN	46000	NaN	NaN	male	no
4	NaN	NaN	61000	NaN	NaN	female	no
5	NaN	NaN	52000	NaN	NaN	male	yes

Figure 2.24. Customer datasets vertical merge in Jupyter Notebook by Arden University. (Accessible code).

Now let's look at horizontal concatenation.

```
In [11]: # We can do the intergration from here
concatenated_df = pd.concat([df1, df2], axis=1)  # Concatenate along rows (horizontally)

# display the concatenated dataframe
concatenated_df
```

Out[11]:

	Age	Owned House	Salary(£)	Marital Status	Distance to the nearest town(km)	Gender	Salary(£)	Car Owner
0	66	yes	54000	widowed	5.4	male	54000	no
1	32	yes	32150	married	10.2	female	32150	yes
2	27	no	29100	married	6.7	non-binary	29100	yes
3	40	no	46000	single	5.7	male	46000	no
4	51	yes	61000	divorced	15.0	female	61000	no
5	22	yes	52000	single	26.0	male	52000	yes

Figure 2.25. Customer datasets horizontal concatenation in Jupyter Notebook by Arden University. (Accessible code).

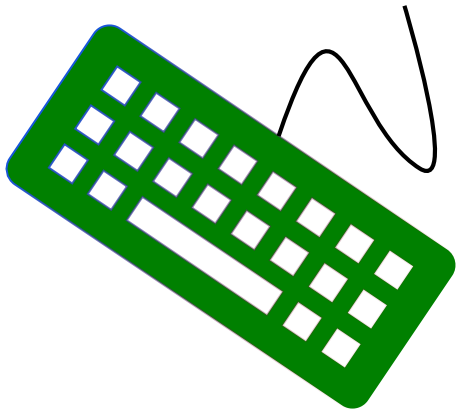
As we can see, by setting (axis = 1), we have concatenated the two dataframes horizontally. However, we can see that two **Salary(£)** attributes have been created. In this case, with similar attributes in both dataframes, ***pd.merge()*** is a better solution to the problem.

Continue

Topic 3 - Activity 1

Practical exercises in data integration (60 minutes)

Implement the examples in this topic using Jupyter Notebook. One way to learn practical skills is by doing them. Run the examples and play around with them. If you have any errors, try to look at the error messages and debug your code accordingly. Seek help from your tutor if needed.



Continue

Topic 3 - Activity 2

Reading

Pages 30-40 of Chapter 2: Data Preprocessing from Data Mining and Predictive Analytics by Larose and Larose

Pages 28-35 of Part 2: Understanding and describing your data from How to Analyze Data by Radcliffe

Continue

TOPIC SUMMARY

In this third topic, we have looked at the data integration process and used some practical techniques to combine datasets. We have looked at pd.merge() and pd.concat() as the two fundamental techniques for data integration. Do not forget to update your cheat sheets.

Now that we have completed this topic, we can proceed to our next topic: data reduction.

Continue to Topic 4

Topic 4: Data reduction

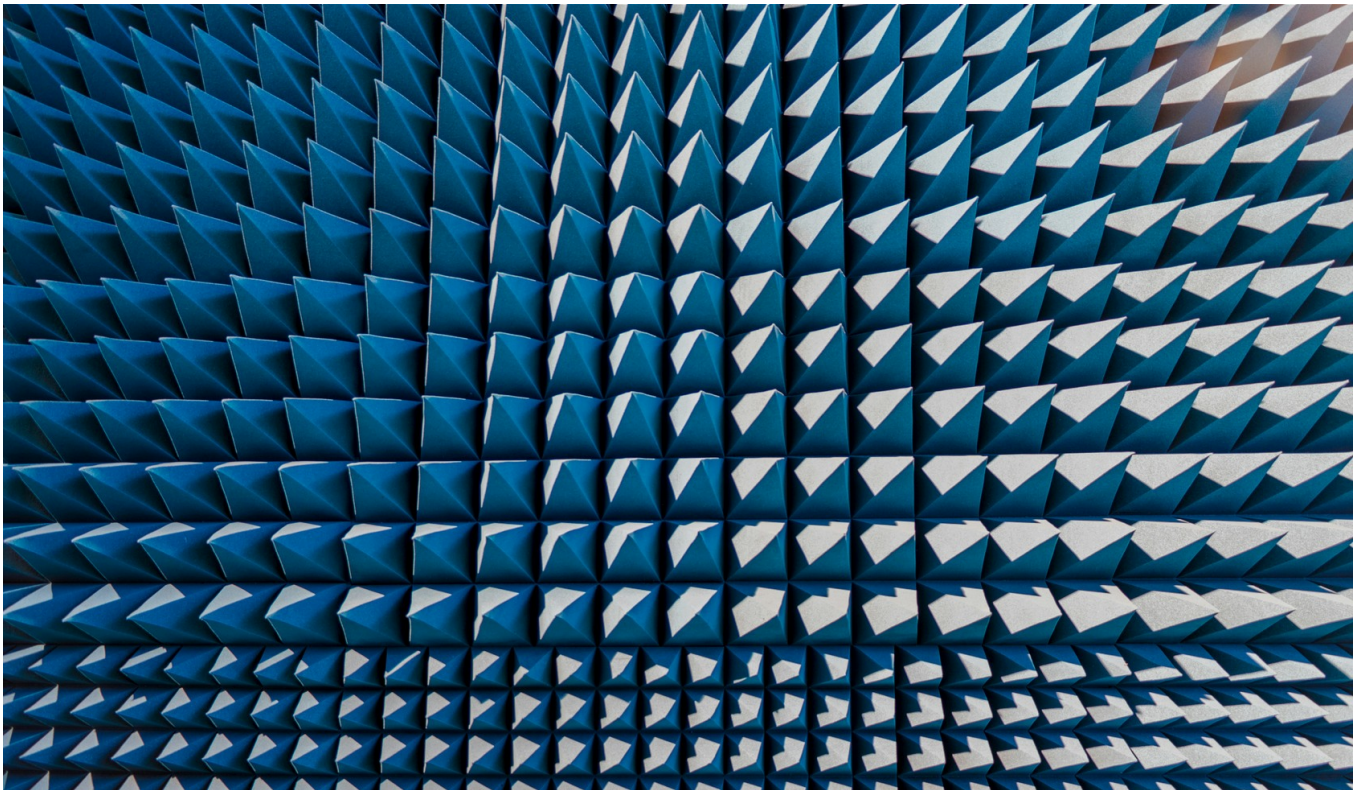


Image from Articulate Rise 360 Content Library.

Did you know?

How long do you think it would take the average person to walk around the world?

After you have thought about it, select 'Reveal'.

Reveal

Image from Articulate Rise 360 Content Library.

If you walked without stopping, it would take the average person around 345 days to walk around the world (Kleanthous, 2024).

Continue

Datasets that we encounter in a real-world domain usually come with various dimensions (number of attributes) and sizes (storage). The purpose of the data reduction process is to reduce or compress the data so that the reduction does not have a negative effect on the quality of the original data.

The reduction can be done on the number of instances or on the dataset's attributes. The following are some of the popular techniques used:

Continue

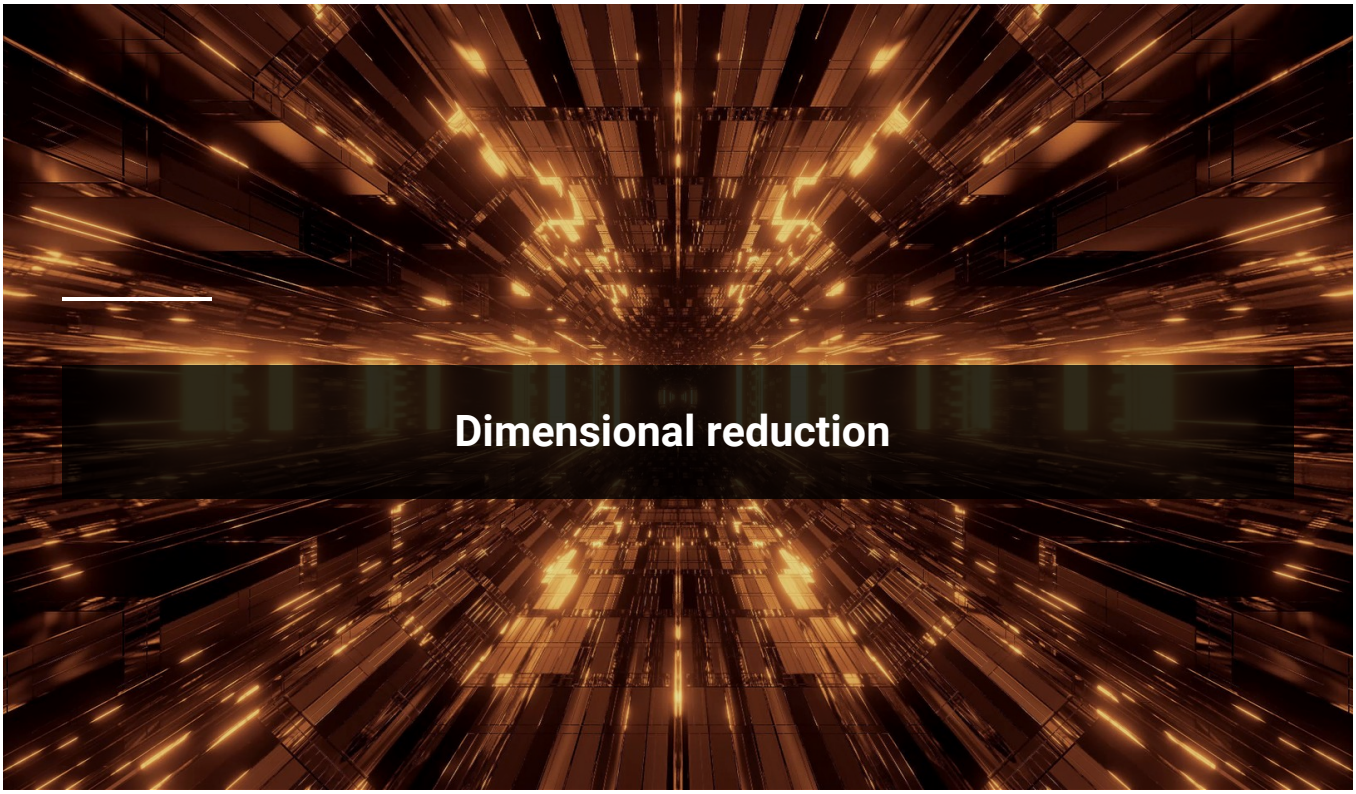


Image from Articulate Rise 360 Content Library.

This technique is probably the most straightforward among the techniques we discuss. In this technique, we reduce the dataset size by removing any redundant instances and clean the data by removing instances that will not be useful in the analysis (e.g., instances with multiple missing attribute values).

For more technical approaches, here are two methods of dimensional reduction:

- **Principal component analysis (PCA)**: Using PCA, an original dataset is imposed onto a smaller space, and the dimension reduction of the dataset can be achieved.
- **Attribute subset selection:** Many attributes to that dataset are redundant with large datasets. What the attribute subset selection does is reduce both the data volume and data dimensionality by eliminating redundant and irrelevant attributes.

Continue

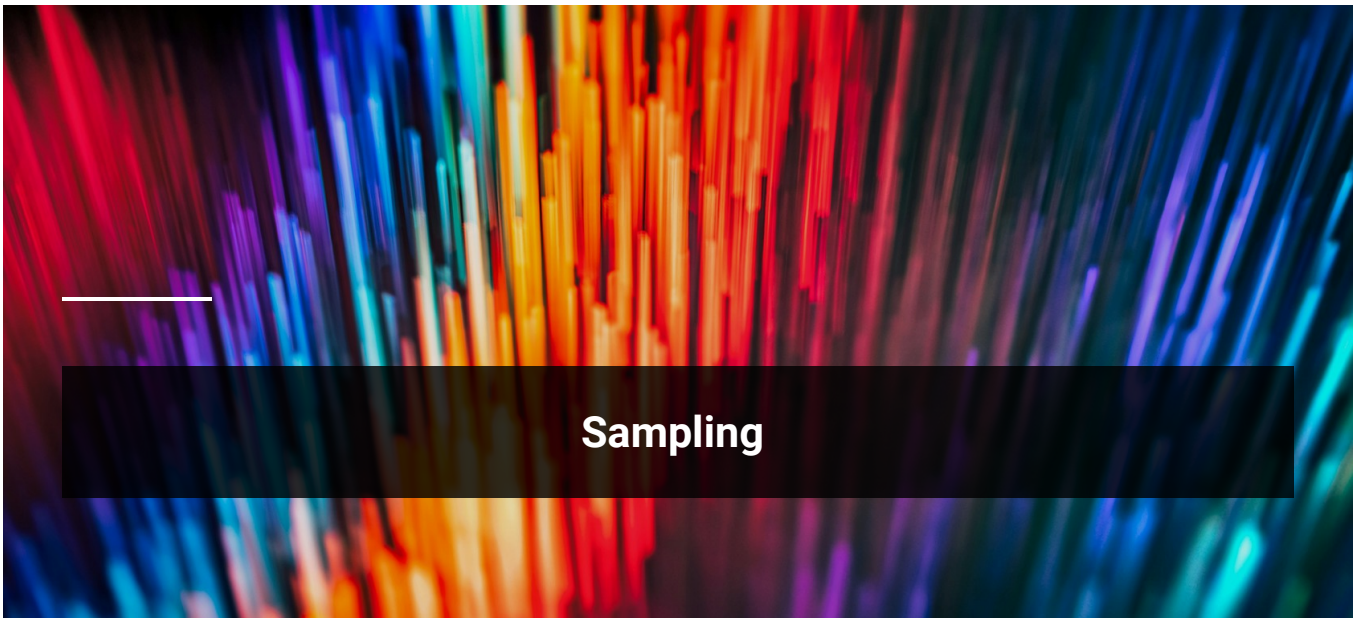


Image from Articulate Rise 360 Content Library.

Another popular approach is to reduce the dimension of a dataset from a large dataset into a smaller dataset sample representation of the original dataset. Some of the sampling techniques are:

SIMPLE RANDOM SAMPLING | STRATIFIED SAMPLING

This is the simplest form, where all instances will have an equal probability of being selected. There are two variations of this method:

- **Sampling without replacement** – As the name suggests, a sample from the population is removed from the population (dataset) as an instance. Each instance can only be selected as a sample once.
- **Sampling with replacement** – An instance will be put back into the population (dataset) after it is selected as a sample. In this case, the same instances can be selected more than once.

SIMPLE RANDOM SAMPLING | STRATIFIED SAMPLING

In its simplest form, this process starts with a pre-specified group. This can be done by grouping the instances based on similar characteristics. For example, if we are trying to sample a dataset of students from different countries, we can start by grouping the students based on the continent they come from. There are two ways we can sample these groups:

- Equal samples are drawn from each group despite the groups being of different sizes.

- The number of samples drawn from each group is proportional to the group size (i.e., using a percentage of the group as the number of samples).

Continue

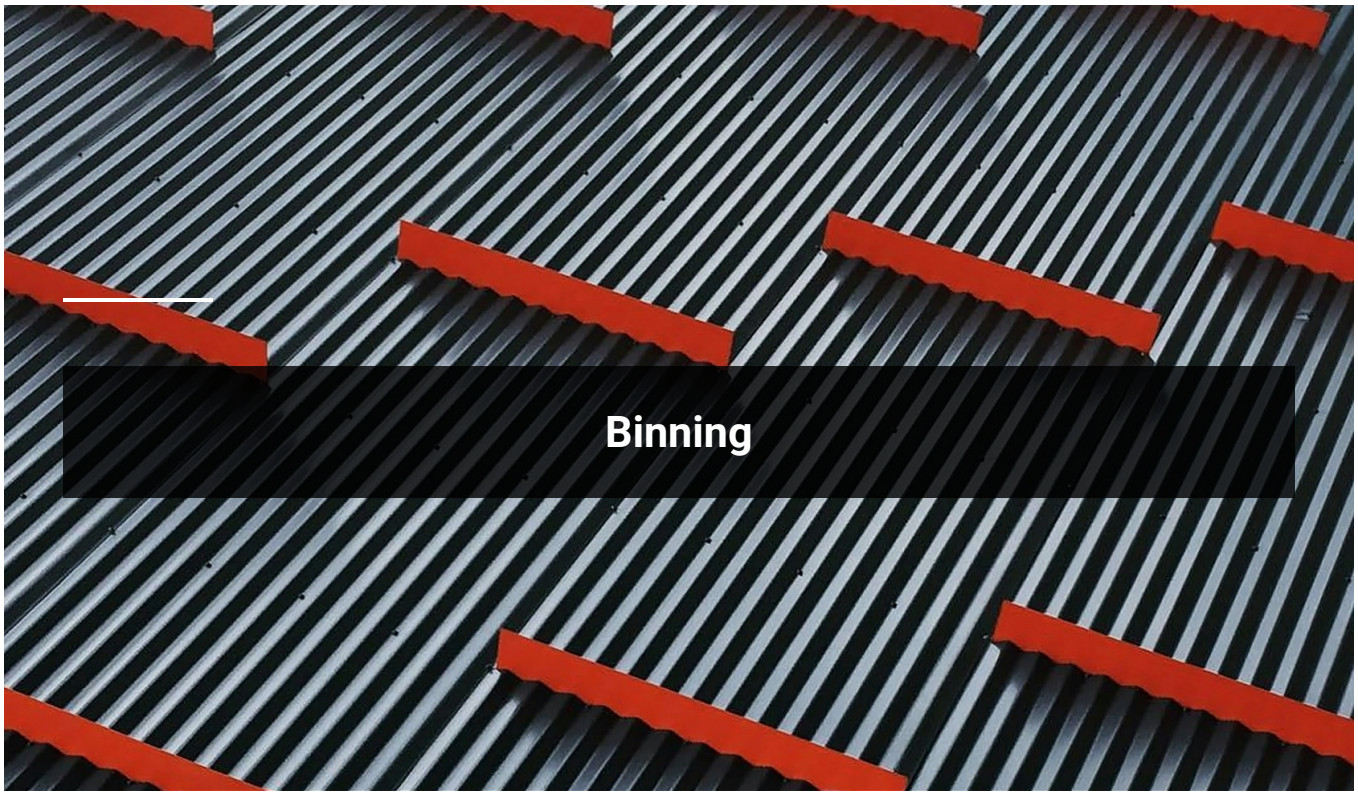


Image from Articulate Rise 360 Content Library.

Data binning or discretisation is used to divide attributes of a continuous nature into data intervals. We can replace many constant values of the attributes by labelling small intervals. This means that mining results are shown in an easily understandable manner.

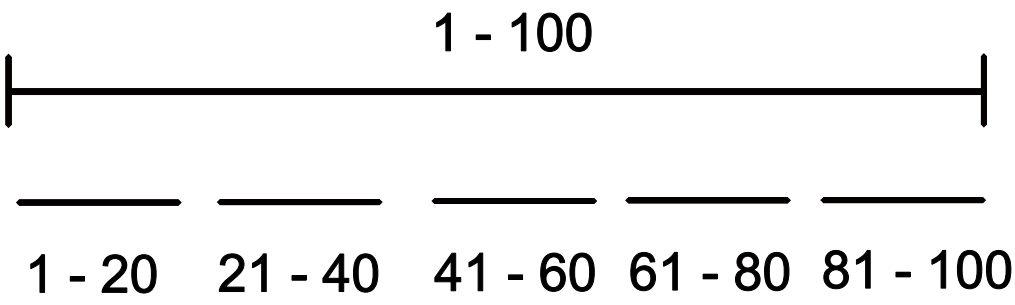


Figure 2.26. Binning by Arden University. (Accessible text).

Now that we understand some of the techniques used in data reduction, let us look at some implementations of these techniques using Jupyter Notebook.

Continue

Example 1: Principal component analysis using iris data

In this example, we will use the iris data we encountered in Lesson 1. The full description of the dataset is available here: Iris by UCI Machine Learning Repository.

Please review the description again, giving you a better understanding of the data.

Step 1: Let us load and examine the iris dataset.

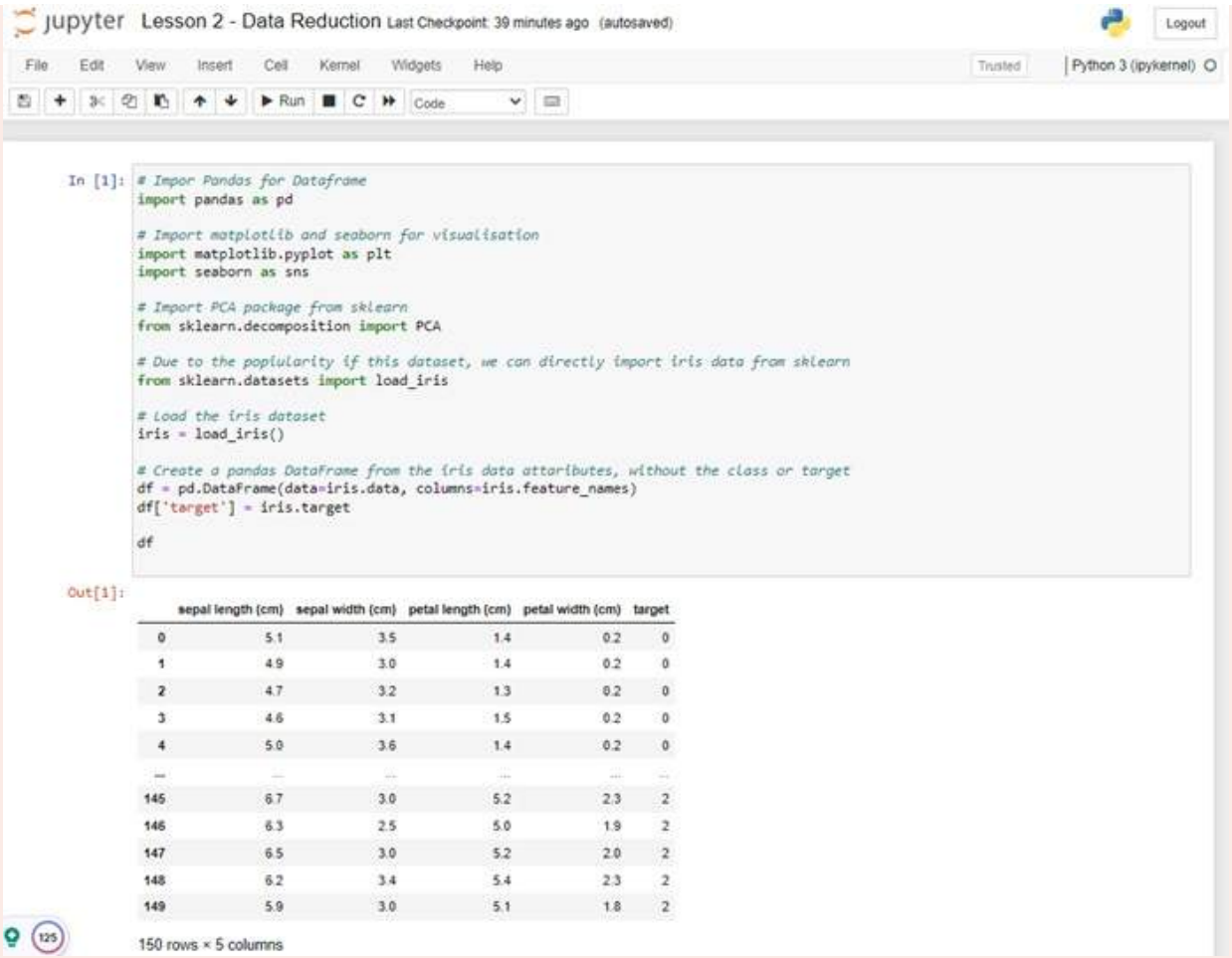


Figure 2.27. Data loading in Jupyter Notebook by Arden University. (Accessible code).

As we can see from the Notebook above, the iris dataset consists of four attributes:

- Sepal length (cm)
- Sepal width (cm)
- Petal length (cm)
- Petal width (cm)

Each instance within this dataset belongs to a certain class of iris. This dataset represents the class using the name **target** with **0 – Iris Sentosa**, **1 – Iris Versicolor** and **2 – Iris Virginica**.

Step 2: Apply PCA to the dataset.

Using PCA, we want to reduce the number of attributes into two components: PC1 and PC2. Here we use two components to plot the final dataset as well.

```
In [4]: # Apply PCA to reduce the dimensionality of the iris data to 2 components
pca = PCA(n_components=2)
components = pca.fit_transform(df)

# Create a new DataFrame with the PCA components and the target variable
pca_df = pd.DataFrame(data=components, columns=['PC1', 'PC2'])
pca_df['target'] = iris.target
pca_df
```

Out[4]:

	PC1	PC2	target
0	-2.865415	0.296295	0
1	-2.892047	-0.183785	0
2	-3.054980	-0.174827	0
3	-2.920230	-0.331582	0
4	-2.906852	0.295917	0
...	...	...	...
145	2.173075	0.127588	2
146	1.784888	-0.443191	2
147	2.005296	0.010566	2
148	2.134042	0.037797	2
149	1.657506	-0.379046	2

150 rows × 3 columns

Figure 2.28. PCA implementation in Jupyter Notebook by Arden University. (Accessible code).

Finally, we can plot the data. The different colours represent the different target classes. We can see that the reduced-dimensional data separates the target classes reasonably well, indicating that the PCA transformation has captured most of the important variability in the original dataset.

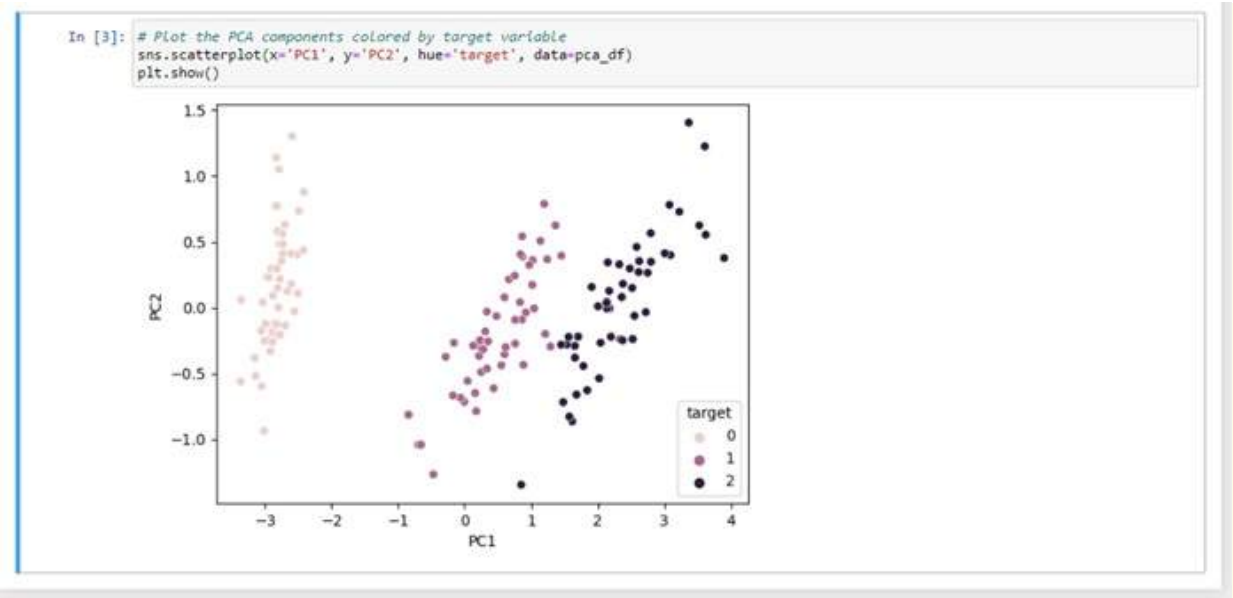


Figure 2.29. PCA plot in Jupyter Notebook by Arden University. (Accessible code).

Continue

Example 2: Sampling

In this example, we will group the iris data into three groups using K-mean clustering, which we will discuss in Lesson 8 of this module. We will then randomly select three samples from each group.

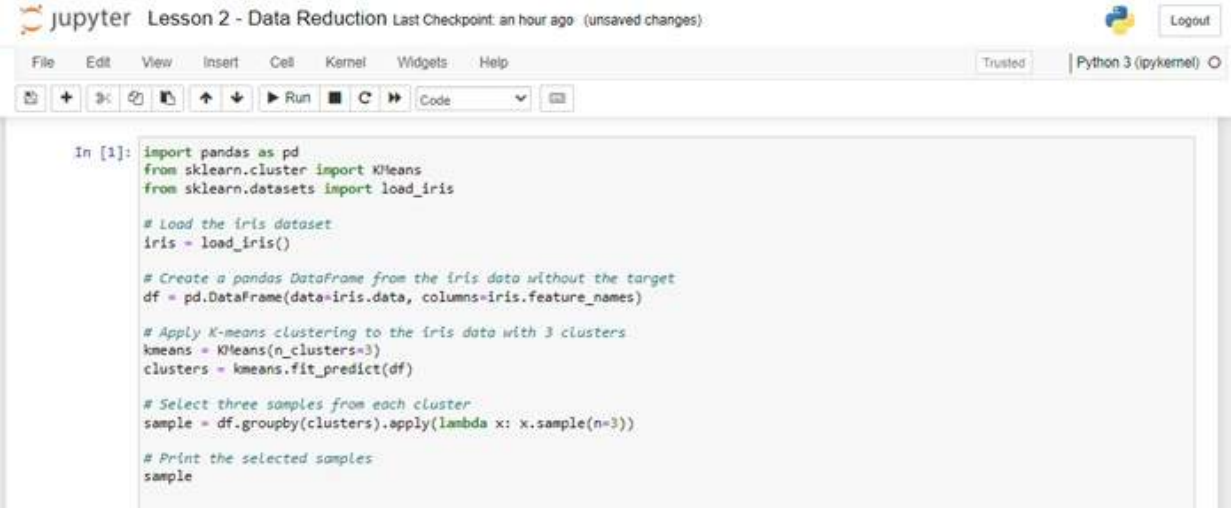


Figure 2.30. Sampling in Jupyter Notebook by Arden University. (Accessible code).

Output:

Out[1]:

		sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	28	5.2	3.4	1.4	0.2
	27	5.2	3.5	1.5	0.2
	5	5.4	3.9	1.7	0.4
1	81	5.5	2.4	3.7	1.0
	87	6.3	2.3	4.4	1.3
	92	5.8	2.6	4.0	1.2
2	116	6.5	3.0	5.5	1.8
	134	6.1	2.6	5.6	1.4
	129	7.2	3.0	5.8	1.6

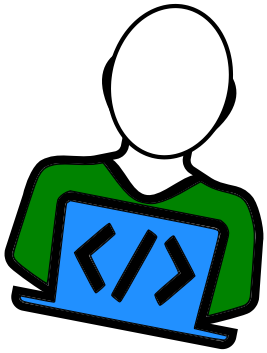
Figure 2.31. Sampling outputs in Jupyter Notebook by Arden University. (Accessible output).

Continue

Topic 4 - Activity 1

Data reduction practice

Implement the examples in this topic using Jupyter Notebook. One way to learn practical skills is by practising them. Run the examples and play around with them. If you have any errors, try to look at the error messages and debug your code accordingly. Seek help from your tutor if needed.



Continue

Topic 4 - Activity 2

Read the following:

Pages 43-62 of Chapter 2: Data from Introduction to Data Mining, Global Edition by Tan *et al.*

Continue

TOPIC SUMMARY

In this topic, we examined some popular data reduction techniques, followed by some practical examples of accomplishing the task. You will now be aware that data reduction aims to define the data more compactly without running the risk of reducing the data quality. You are encouraged to read the chapter on this topic in the textbook.

Now that we have covered this topic, it is time for us to proceed to the final topic of the lesson: data transformation.

Continue to Topic 5

Topic 5: Data transformation

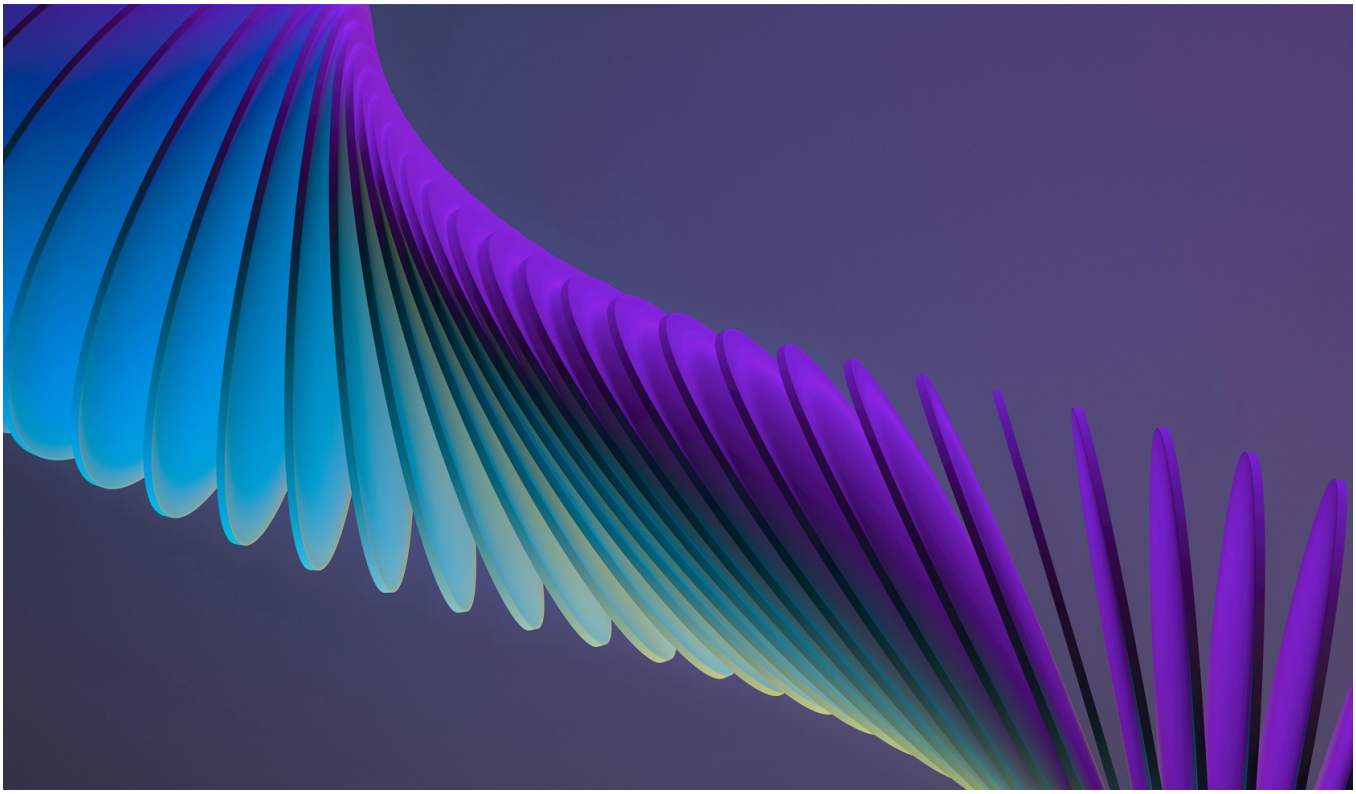


Image from Articulate Rise 360 Content Library.

Data transformation is a technique used to convert raw data into a suitable format that efficiently retrieves information, as raw data can often be difficult to understand. The process includes data cleaning and reduction to convert the data into an appropriate form, which we covered in the previous topic. Doing so will allow patterns in the analysis to become much clearer and easier to understand. During the data transformation process, an analyst will determine the structure of the data, meaning that data transformation may be either:

Constructive: adds, copies, or replicates data

Destructive: deletes fields or records

Structural: renaming, moving, or combining columns.

Continue

Image from Articulate Rise 360 Content Library.

The user can apply several data transformation techniques to improve data structure and clean up data before analysis. The most common techniques are outlined below.

Data smoothing

Data smoothing is a process that removes “noise” from the dataset using algorithms. This helps with pattern prediction because large datasets can often be difficult to digest when determining patterns. Common processes in smoothing are:

- **Binning:** Splits the sorted data into several bins and smooths the data values in each bin, considering the neighbouring values around it. This is very similar to the binning process found when constructing histograms.
- **Regression:** Identifies the relation between two dependent attributes, so one attribute can be used to predict the other.
- **Clustering:** Groups of similar data values to form a cluster. The values that lie outside a cluster are known as outliers.

Continue

Attribute construction

The attribute construction method introduces new attributes to an existing dataset, thereby constructing a new dataset that eases data mining. This simplifies the original data and allows the mining to be more efficient.

For example, a property developer analyst has a dataset referring to height and width measurements of different buildings. Here, they can construct a new attribute called area from the existing attributes *height* and *width*. Doing so will help us understand the attributes’ relations much more clearly.

Image from Articulate Rise 360 Content Library.

Continue

Data normalisation

Normalising the data refers to scaling the data values to a much smaller range, such as [-1, 1] or [0.0, 1.0]. This is a useful tool in an analyst's arsenal. There are different methods to normalising the data, and we consider the most common technique below.

Min-max normalisation: This method implements a linear transformation on the original data. Let us consider that we have *minA* and *maxA* as the minimum and maximum values observed for attribute *A*, and V_i is the value for attribute *A* that is to be normalised.

The min-max normalisation would map V_i to the V_i' in a new smaller range [new_minA, new_maxA]. Therefore, the formula for the min-max normalisation would be:

$$v_i' = \frac{v_i - min_A}{max_A - min_A}(new_{maxA} - new_{minA}) + new_{_minA}$$

Figure 2.32. Min-max normalisation by Arden University.

For example, we have £18,000 and £75,000 as the minimum and maximum values for the attribute income, and [0.0, 1.0] is the range in which we have to map a value of £64,600.

The value £64,600 would be transformed using min-max normalisation as follows:

$$v_i' = \frac{64600 - 18000}{75000 - 18000}(1.0 - 0.0) + 0.0 = 0.817$$

Figure 2.33. Min-max normalisation calculations by Arden University.

Continue

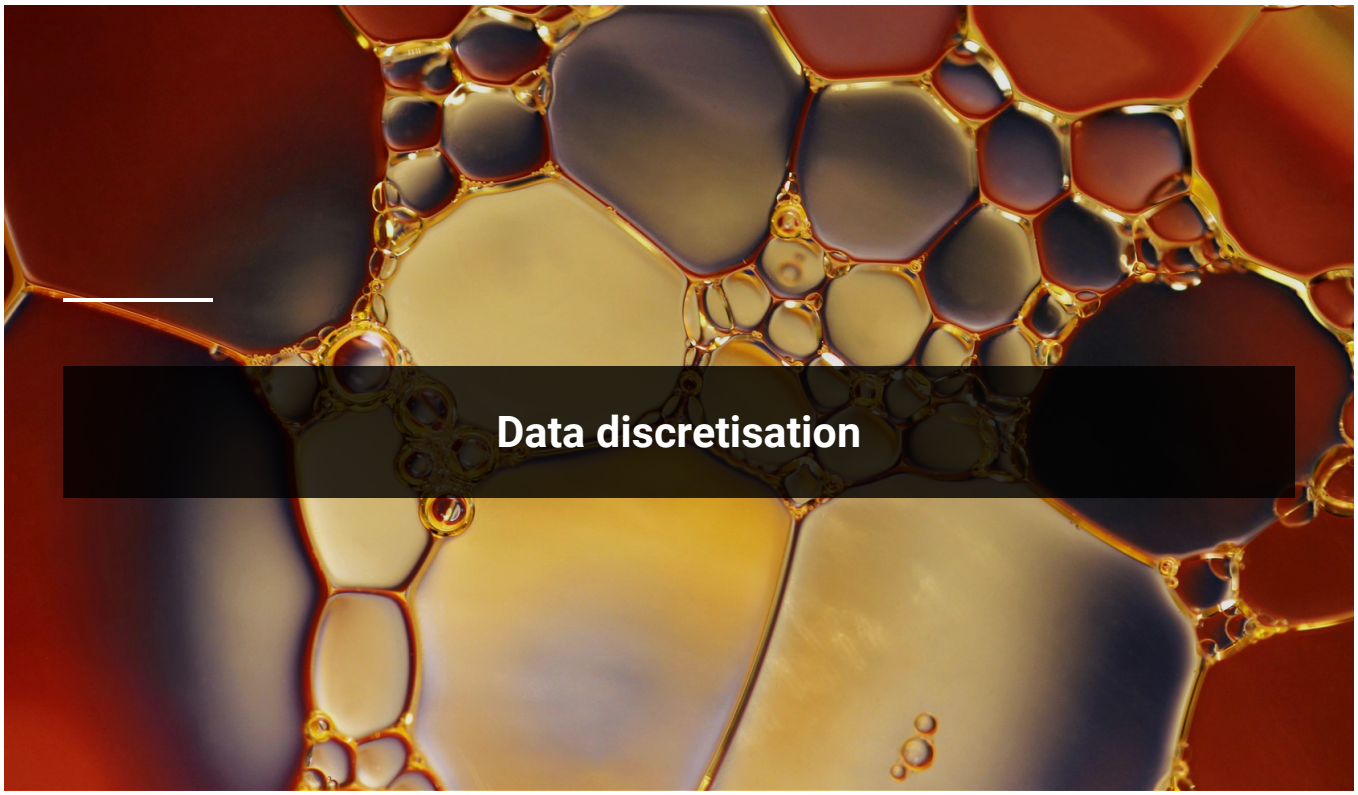


Image from Articulate Rise 360 Content Library.

This is a process of converting continuous data into a set of data intervals. Small interval labels substitute continuous attribute values. This makes the data easier to study and analyse. If a data mining task handles continuous attributes, then its discrete values can be replaced by constant quality attributes, improving the efficiency of the task.

Example 1 – Min-max normalisation using *MinMaxScaler()*

Let us look at the following Jupyter Notebook on min-max normalisation:

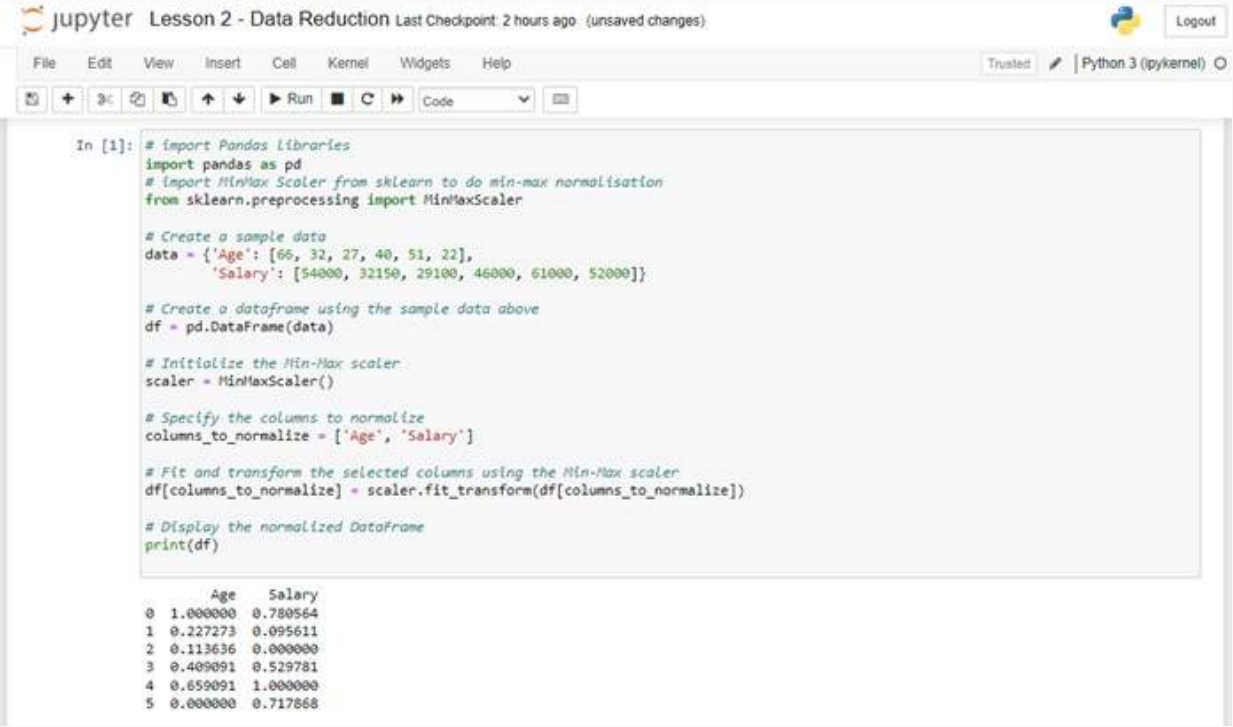


Figure 2.34. Min-max normalisation implementation in Jupyter Notebook by Arden University. (Accessible code).

Example 2 – Create new attributes using *LabelEncoder()*

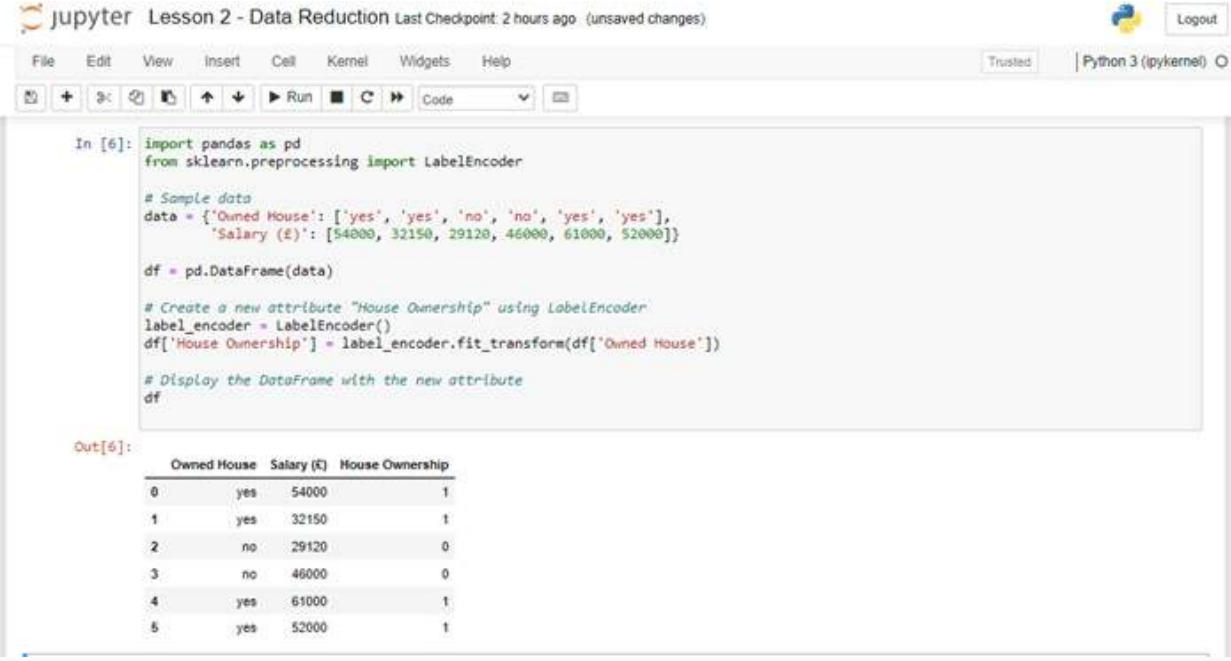


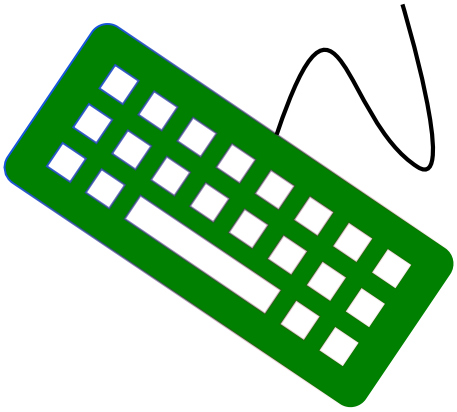
Figure 2.35. Label encoder implementation in Jupyter Notebook by Arden University. (Accessible code).

Continue

Topic 5 - Activity 1

Min-max normalisation practical

Using a dataset of your choice, ideally one with a combination of numerical and discrete attributes, implement min-max normalisation on the numerical attributes and create new attributes using existing ones.



Continue

Topic 5 - Activity 2

Read the following:

Pages 40-53 of Chapter 2: Data Preprocessing from Data Mining and Predictive Analytics by Larose and Larose

Pages 22-30 of Chapter 1: Liftoff! Prediction Takes Action from The Big Data-Driven Business: How to Use Big Data to Win Customers, Beat Competitors, and Boost Profits by Glass and Callahan

Continue

TOPIC SUMMARY

In this final topic, we have looked at various strategies for data transformation. The practical examples show how we can change the structure and dimension of the data values and create new attributes from existing ones.

Continue to Knowledge check

Knowledge check

Answer the following multiple choice questions

Question

01/03

What process is needed when you have more than one source of data needed for analysis? Please choose the correct answer.

- ◯ Data reduction
- ◯ Data cleaning
- ◯ Data integration
- ◯ Data transformation

Question

02/03

What does NaN mean in relation to missing data? Please choose the correct answer.

- ◯ Not a number
- ◯ Not a numerical value
- ◯ Not again nan
- ◯ Never ask nonsense

Question

03/03

What is the purpose of data cleaning? Which of the following is true?

- [ ] Data cleaning removes or replaces any missing or redundant values in a dataset.

- [ ] Data cleaning is used to create new attributes and provide more value to the attributes.

Lesson summary

LESSON SUMMARY

Congratulations, you have completed Lesson 2!

Raw data is a constant worry for a data scientist. Sometimes, when we receive the raw data, it is quite normal to be shocked by the state of the data. Hence, data scientists usually spend about 80% of their time on the processes we discussed in this lesson: **data cleaning, data integration, data reduction and data transformation**.

After the first stage of cleaning the data, removing or replacing duplicate, redundant and missing data, we looked at some techniques of data integration. We then focused on compressing and condensing our data meaningfully without the risk of losing the information and quality of the data. Finally, we discussed data transformation and how this process allows us to convert our data into a better representation and even to create new attributes that might enhance our data quality.

Now that we have covered all aspects needed for data, the next lesson will focus on the first machine-learning algorithm: regression.

Continue to 'Reading list'

Reading List

All reading and resources in this module are provided by the Arden University Library service. The full reading list for this module can be accessed at the link below. More books, journals and databases can be found on the Library Portal. If you need these resources in a different format, our accessibility tools can help.

Full reading list

You can find all the key, essential and wider reading and resources for this module on the dedicated reading list page on the Learning Success Hub.

VIEW READING LIST

References

Cited sources that do not appear in your reading list:

BasuMallick, C., (2021). Why Machine Learning Accuracy Matters and Top Tools to Supercharge It. *Spiceworks* [online], 24 August 2021. Austin, TX: Spiceworks.

Kleanthous, B., (2024). How Long Would it Take to Walk Around the World? *The Calculator Site* [online], 26 April 2024. London: Hazell Industries.

LinkedIn, (2023). *machine learning in Worldwide* [webpage, online]. Sunnyvale, CA: LinkedIn.

Rençberoğlu, E., (2019). Fundamental Techniques of Feature Engineering for Machine Learning. *Towards Data Science* [online], 1 April 2019.